



TM293

# **Automation Studio Target for Simulink**

**Requirements**

|                  |                                                                                                               |
|------------------|---------------------------------------------------------------------------------------------------------------|
| Training modules | TM210 - Working with Automation Studio<br>TM213 - Automation Runtime<br>TM223 - Automation Studio diagnostics |
| Software         | Starting with MATLAB R2017b<br>C/C++ compiler<br>Automation Studio 4.x.x or later                             |
| Hardware         | None                                                                                                          |

# Table of contents

|                                                             |    |
|-------------------------------------------------------------|----|
| 1 Introduction.....                                         | 5  |
| 1.1 Learning objectives.....                                | 6  |
| 1.2 Symbols and safety notices.....                         | 6  |
| 2 General information.....                                  | 7  |
| 2.1 Simulation.....                                         | 7  |
| 2.2 Model-based development.....                            | 8  |
| 2.3 Virtual commissioning.....                              | 10 |
| 3 MATLAB.....                                               | 11 |
| 3.1 User interface.....                                     | 11 |
| 3.2 Help documentation.....                                 | 11 |
| 3.3 Simulink.....                                           | 13 |
| 4 Automation Studio Target for Simulink.....                | 15 |
| 4.1 Installation and registration.....                      | 16 |
| 4.2 Help documentation.....                                 | 19 |
| 5 My first project.....                                     | 23 |
| 6 Principle of automatic code generation.....               | 24 |
| 7 Interface to Automation Studio - B&R config block.....    | 26 |
| 7.1 Basic settings.....                                     | 26 |
| 7.2 Automatic transfer.....                                 | 27 |
| 7.3 External mode.....                                      | 28 |
| 7.4 Cycle time and task class.....                          | 30 |
| 7.5 Dependence on additional files.....                     | 31 |
| 7.6 Additional functionality.....                           | 31 |
| 8 Variables, constants and arrays.....                      | 33 |
| 8.1 Code generation.....                                    | 33 |
| 8.2 B&R input block.....                                    | 35 |
| 8.3 B&R output block.....                                   | 37 |
| 8.4 Creating internal variables and constants.....          | 37 |
| 8.5 Creating arrays.....                                    | 39 |
| 9 Creating user libraries.....                              | 43 |
| 10 Structures.....                                          | 46 |
| 10.1 B&R bus block.....                                     | 46 |
| 10.2 Example: Bus block.....                                | 47 |
| 10.3 B&R struct block.....                                  | 47 |
| 10.4 Example: Structure block.....                          | 49 |
| 10.5 B&R extended struct block.....                         | 49 |
| 10.6 Example: Extended Structure block.....                 | 50 |
| 11 Automation Studio hot plugging interface.....            | 52 |
| 12 Example: Automation Studio Hot-Plug Interface block..... | 56 |
| 13 Model structuring and data management in MATLAB.....     | 57 |
| 13.1 Subsystems.....                                        | 57 |

|                                        |    |
|----------------------------------------|----|
| 13.2 Example: Subsystems.....          | 58 |
| 13.3 Model referencing.....            | 58 |
| 13.4 Model referencing example.....    | 60 |
| 13.5 Data dictionary.....              | 60 |
| 13.6 Data dictionary example.....      | 62 |
| 14 Summary.....                        | 63 |
| 15 Appendix.....                       | 64 |
| 15.1 Programming interfaces (API)..... | 64 |
| 15.2 Data type conversions.....        | 64 |
| 15.3 Diagnostics.....                  | 66 |

# 1 Introduction

For years, the **MathWorks** software package from **MATLAB®** (<http://www.mathworks.com>) has served as a powerful tool for solving engineering, mathematical and business problems and is widely used in the industrial world. MATLAB® is a numerical computing environment and programming language. The greatest strength of the program is its ability to handle large matrices, as the name **MATrix LABoratory** suggests. MATLAB® can be extended with various add-on packages, such as **Simulink®**. This program package enables graphical creation of simulation models that allow complex technical processes to be adapted under realistic conditions.

Automatic implementation of Simulink models in C/C++ code, which has been specially optimized for use in B&R target systems, offers developers new possibilities for designing sophisticated simulation models and control structures that would otherwise be impossible or very time consuming to implement.

The greatest advantages of **automatic code generation** are available to developers who already use MATLAB® and Simulink® for simulation and solution design, as well as to developers who, in the past, have worked hard to revise implemented structures in a language supported by Automation Studio. In the procedures listed below, the tool for automatic code generation provided by B&R represents an innovation with endless possibilities that contribute to productive transformation of control system development.

The basic principle is simple: The module created in Simulink® is automatically translated into the optimal language for the B&R target system using **Simulink Coder®** or **Embedded Coder®** (optional) to ensure maximum performance of the generated source code. Seamless integration into an **Automation Studio project** helps to achieve the ideal development process.

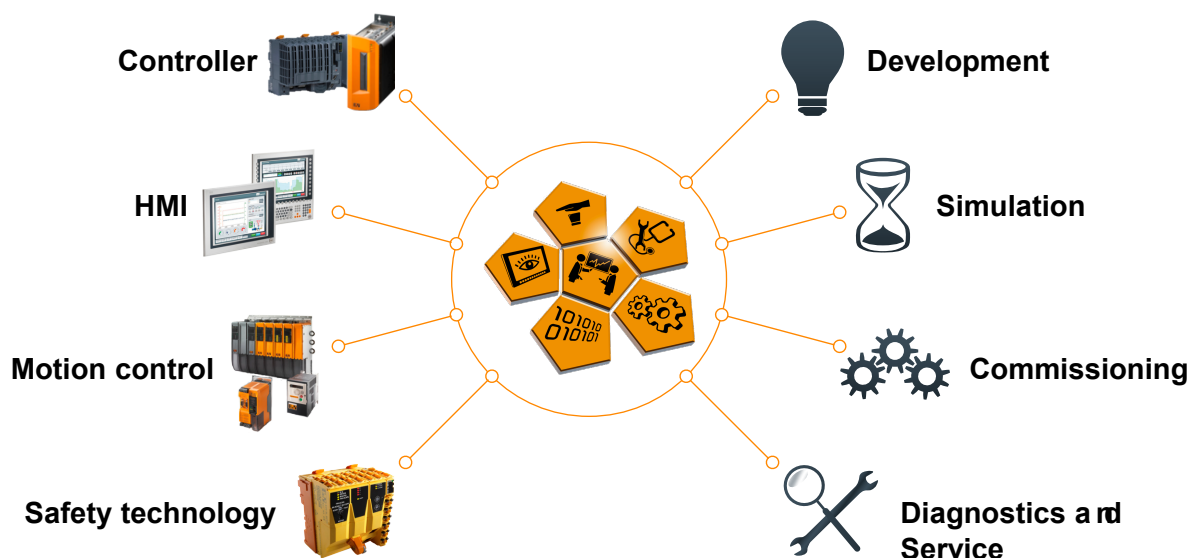


Figure 1: Automation Studio: One engineering tool for the machine's entire lifecycle

## 1.1 Learning objectives

This training module gives participants insight into the basics of simulation and how to use the Automation Studio Target for Simulink interface.

Participants will learn the following:

- How to use Automation Studio Target for Simulink.
- How to use the user interface for the B&R Simulink blocks.
- How to create smaller projects in Simulink.
- How to use automatic code generation.
- About the relationship between the Simulink model and the program generated in Automation Studio.
- How to configure communication between a Simulink model and an Automation Studio project.

## 1.2 Symbols and safety notices

Unless otherwise specified, the symbol descriptions and safety notices listed in "TM210 - Working with Automation Studio" apply.

## 2 General information

The sharp increase in production requirements, the expanding complexity of systems and the development of greater numbers of technical products with software all make **simulation** an essential component of the development process. The following image shows the typical B&R **simulation levels**, which differ in their level of detail. A distinction is made between the **automation hardware level**, the **component and machine level** and the **process and plant level**.

Starting at the base, you have simulation of hardware and software components. Automation Studio supports all possible simulation options.

In the higher level, dynamic processes of machines and systems can be simulated. For this, B&R offers connection to the most popular simulation tools on the market. The third level enables simulation of complex system processes.

Even for process simulation, B&R offers corresponding interfaces for external software in Automation Studio.

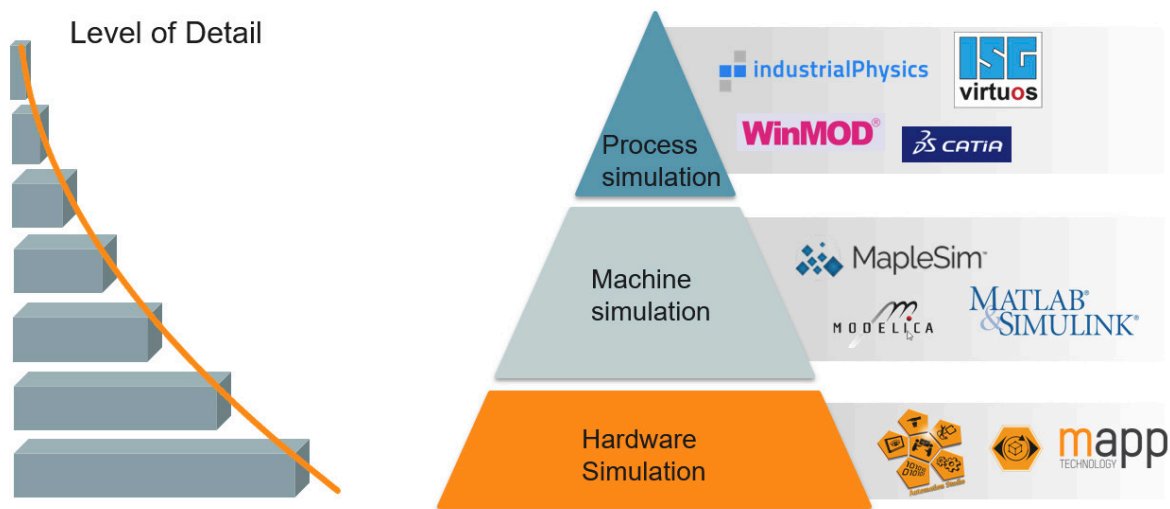
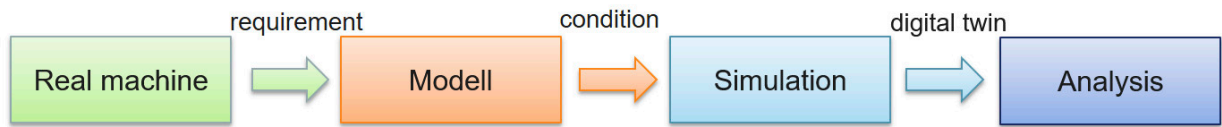


Figure 2: Simulation levels - Plants and processes, machines and components, hardware

### 2.1 Simulation

**Simulation** is an experimental approach to determine static and dynamic properties of a system based on a model. The model makes it possible to gain knowledge about the system and its behavior. This knowledge can then be transferred to reality. For the digital representation of the original system, it is irrelevant whether the system to be examined already exists or is still in planning stage. The image below shows a simplified procedure for creating a simulation model.



## Wind power station

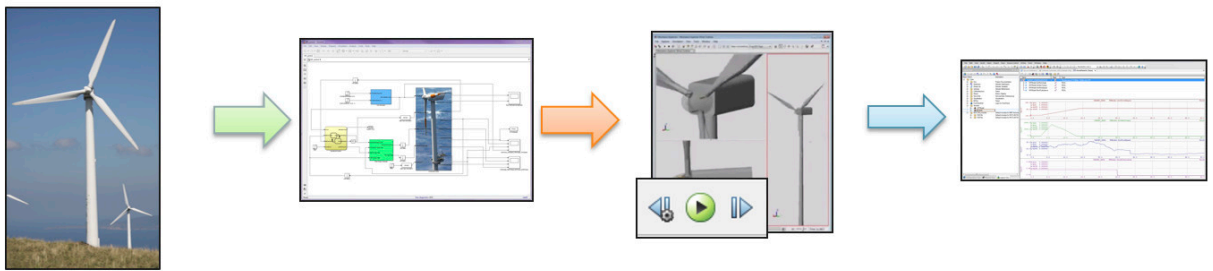


Figure 3: Simplified representation of a simulation.

Customized Simulink models are used as the simulation model in the following.

### Advantages of simulation with MATLAB/Simulink for software and machine development

- Possibility to analyze extremely extensive or complex systems
- Generation of highly efficient program code
- More precise adjustment when presetting system parameters
- Minimization and early detection of errors
- High efficiency and productivity
- High product quality
- Time and place for innovation
- Rapid prototyping
- Reusability
- Accelerated time to market

## 2.2 Model-based development

**Model-based development** is based the principle of simulation. A complex process is first represented in an abstract, realistic model. It is important that all processes necessary for development are contained in the simulation model since any further development is based on it. During the test phase, continuous improvements can be made based on the test results from the model. This process is displayed in the following diagram.



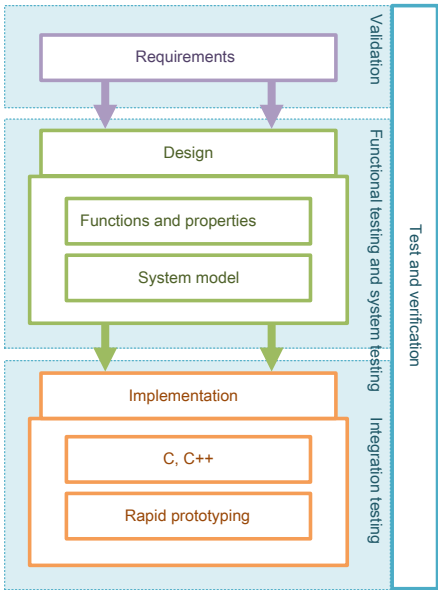


Figure 4: Procedure of model-based development

## 2.3 Virtual commissioning

**Virtual commissioning** refers to the testing of the simulation model in a virtual environment in order to simulate real commissioning as accurately as possible. In most cases of virtual commissioning, the simulation model is a virtual image of a real machine. During virtual commissioning, data is imported, tested and changed on the virtual image of the machine before the software is transferred to the real machine. An executable model enables the software to be tested on a workstation computer or on a real-time system at an early stage.

An important aspect during virtual commissioning of mechanical processes is the 3D representation that simulates machine behavior and thus provides visual feedback for the tester.

It should be noted, however, that virtual commissioning does not replace real commissioning completely. In model-based development, two test procedures have proven to be effective: **Software-in-the-loop (SiL)** and **hardware-in-the-loop (HiL)**.

### 2.3.1 Software-in-the-loop (SiL)

**Software-in-the-loop** is a test procedure used for models in a simulation environment. During a software-in-the-loop simulation, the software that has been developed and the simulation environment are executed on the same hardware. The software communicates with the simulation, which is running on the same processor, via global variables or variable mapping.

### 2.3.2 Hardware-in-the-loop (HiL)

**Hardware-in-the-loop** is a test procedure for evaluating a simulation model of a machine on the real hardware used during commissioning. A real-time simulation is used for the test system in order to represent a controlled system for a closed-loop controller as accurately as possible. An HiL system provides the controller with all input and output signals that would exist in the real environment.

## 3 MATLAB

MATLAB (MATrix LABoratory) offers a programming language that is primarily designed for numerical calculations based on matrix-based mathematics. A wide range of applications is provided by the combination of this development process and a desktop environment for iterative analysis. MATLAB can be extended with a variety of toolboxes, such as Simulink.



For further information, see [MathWorks](https://mathworks.com).  
https://mathworks.com

### 3.1 User interface

The MATLAB user interface consists of five windows. At this point, only a rudimentary description will be provided. For more detailed information, see [Matlab Getting started](https://mathworks.com/help/matlab/getting-started-with-matlab.html) on the MATLAB website:  
[mathworks.com/help/matlab/getting-started-with-matlab.html](https://mathworks.com/help/matlab/getting-started-with-matlab.html)

#### User interface

- 1) **Current folder:** Displays the content of the folder at the current path.
- 2) **Command window:** The command window is used to execute functions and statements.
- 3) **Workspace:** The workspace lists the currently used variables of all data types until the next clean up.
- 4) **Current path:** Displays the current path and offers recently used paths through the drop-down menu.
- 5) **Toolbar:** Quick start for possible functions in MATLAB

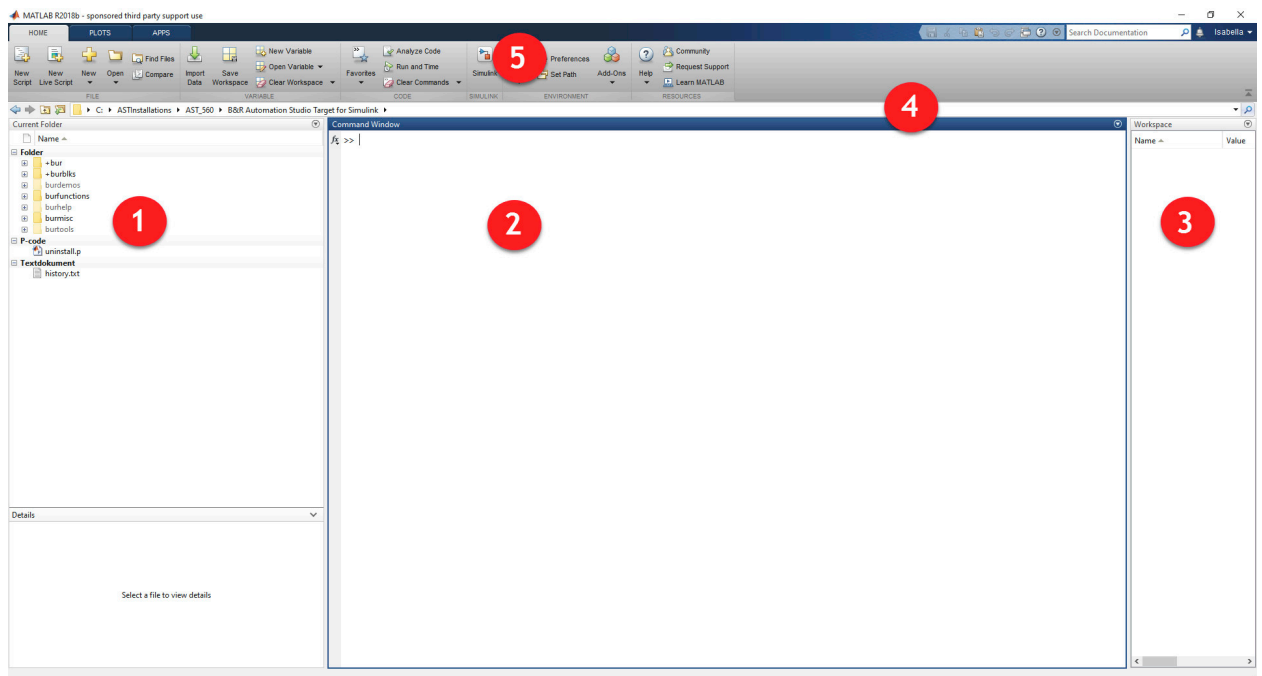


Figure 5: MATLAB user interface

### 3.2 Help documentation

The MATLAB help documentation can be opened via the toolbar. The help documentation contains information about functions, blocks and toolboxes.

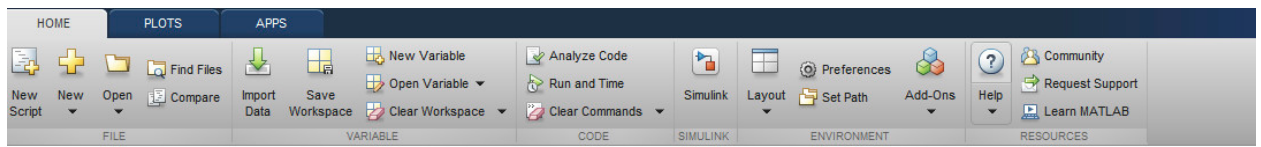


Figure 6: MATLAB's help documentation can be opened via the toolbar.

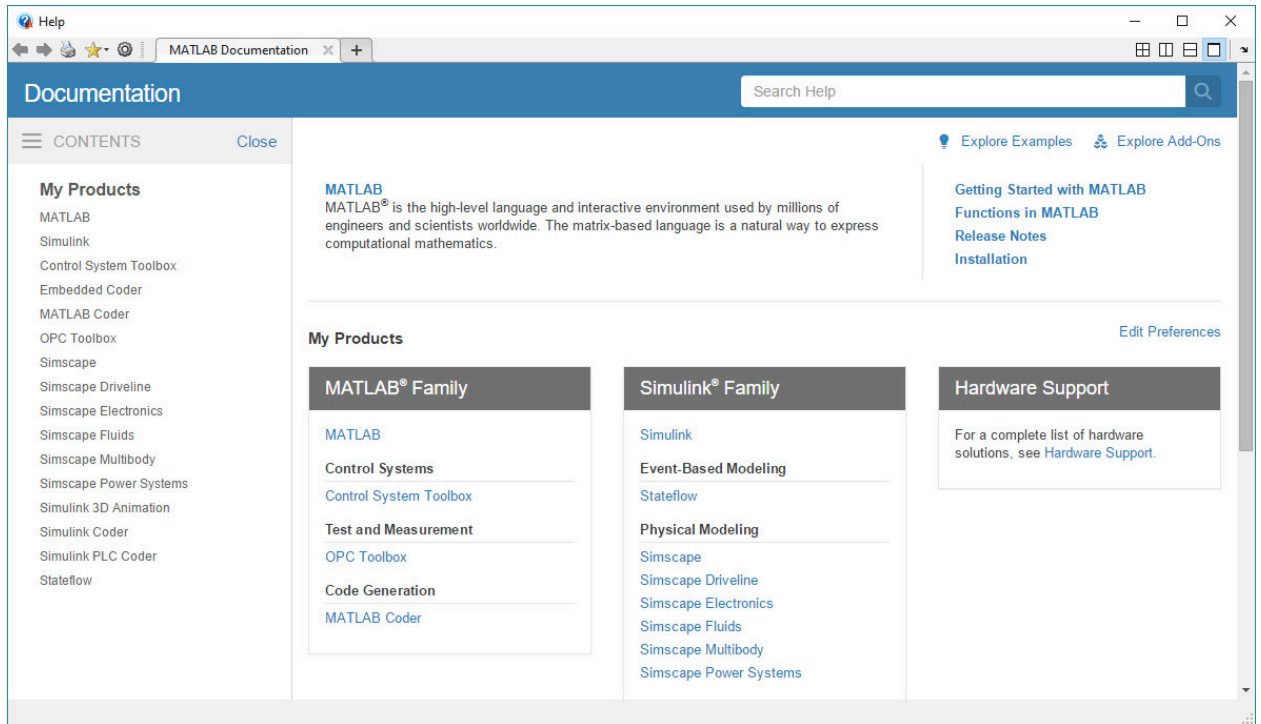


Figure 7: MATLAB help documentation

### 3.3 Simulink

**Simulink** adds graphical programming to MATLAB, providing the foundation for multi-domain simulation and model-based development with MATLAB. This supports system-level design and simulation and enables automatic code generation and continuous testing and verification of embedded systems. In addition, existing and custom MATLAB algorithms can be used in Simulink and the simulation results can be analyzed and processed in MATLAB.

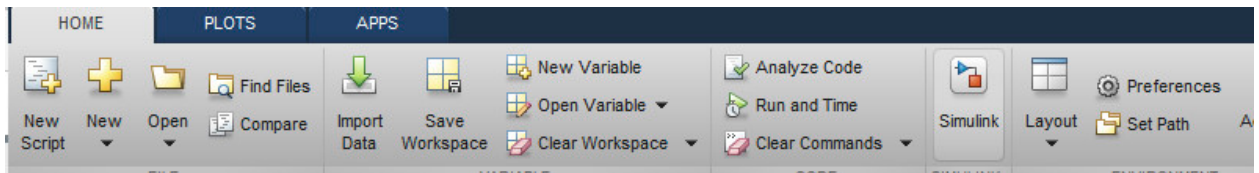


Figure 8: Opening Simulink from the toolbar

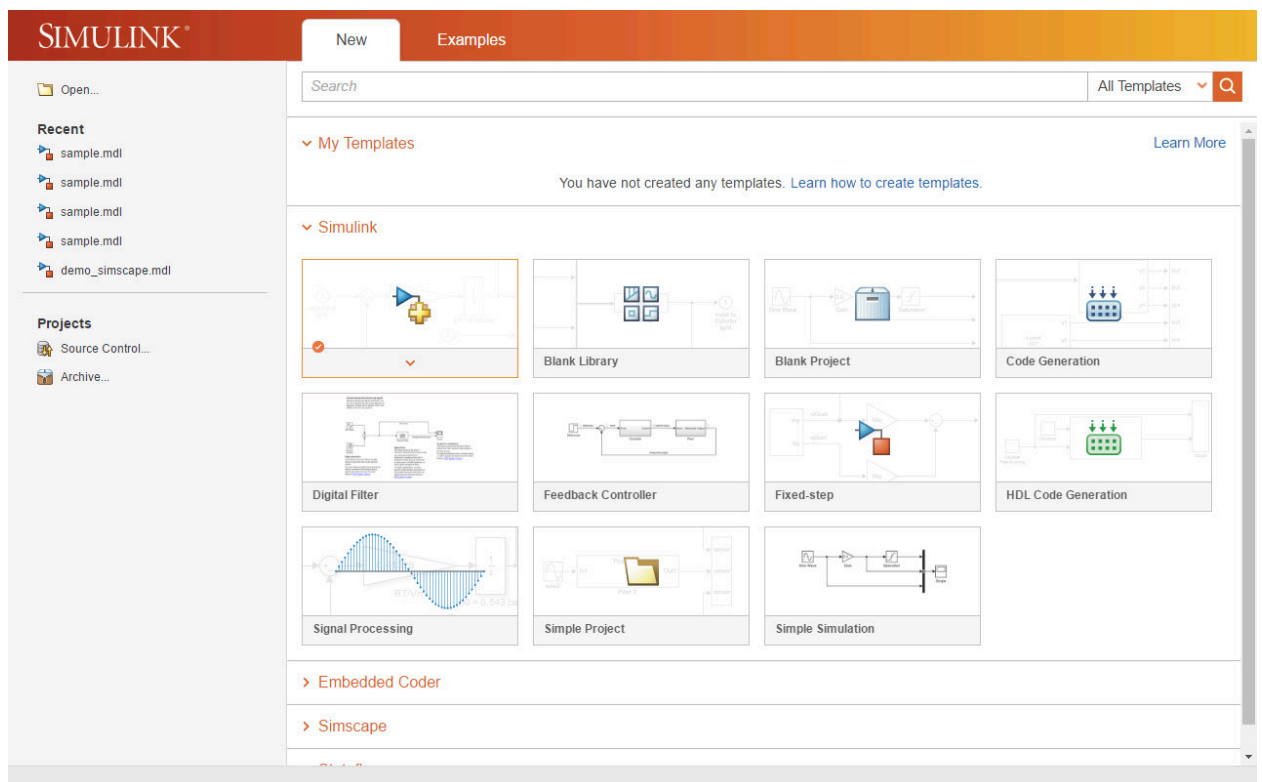


Figure 9: Simulink start page



For further information, see [Simulink - Getting Started](https://mathworks.com/products/simulink.html).  
<https://mathworks.com/products/simulink.html>

#### 3.3.1 Toolbox system

Simulink consists of a **toolbox system** and includes a graphics editor, custom block libraries and solvers for modeling and simulating dynamic systems. Simulink libraries intended for specific application areas can be added to Simulink individually. Automation Studio Target for Simulink is another toolbox with blocks that are added to the Simulink library.

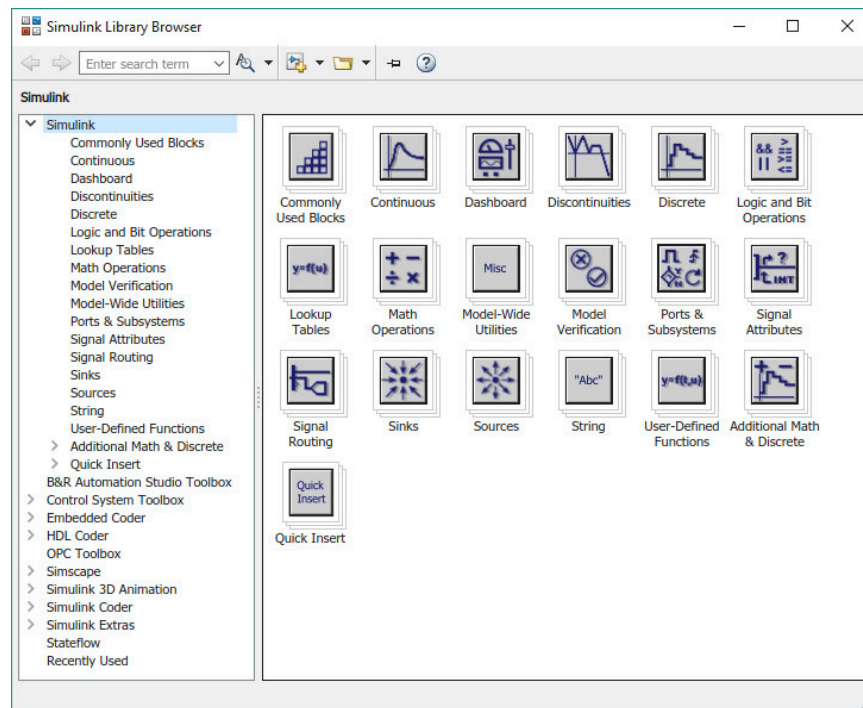


Figure 10: Simulink library

## 4 Automation Studio Target for Simulink

**Automation Studio Target for Simulink** is a toolbox for extending Simulink and serves as an interface between MATLAB/Simulink and Automation Studio.

Installing Automation Studio Target for Simulink adds a B&R library to the Simulink library. It provides blocks that allow a connection to Automation Studio and seamless integration of the generated program code. A detailed explanation of the individual blocks is provided in the following sections. Automation Studio Target for Simulink supports the creation of variables, constants, one-dimensional and multidimensional fields and structures.

In addition to B&R-specific blocks, Automation Studio Target for Simulink also supports all standard input/output blocks from Simulink. In addition to the standard MATLAB setting options, however, the B&R blocks also offer settings adapted to Automation Studio.

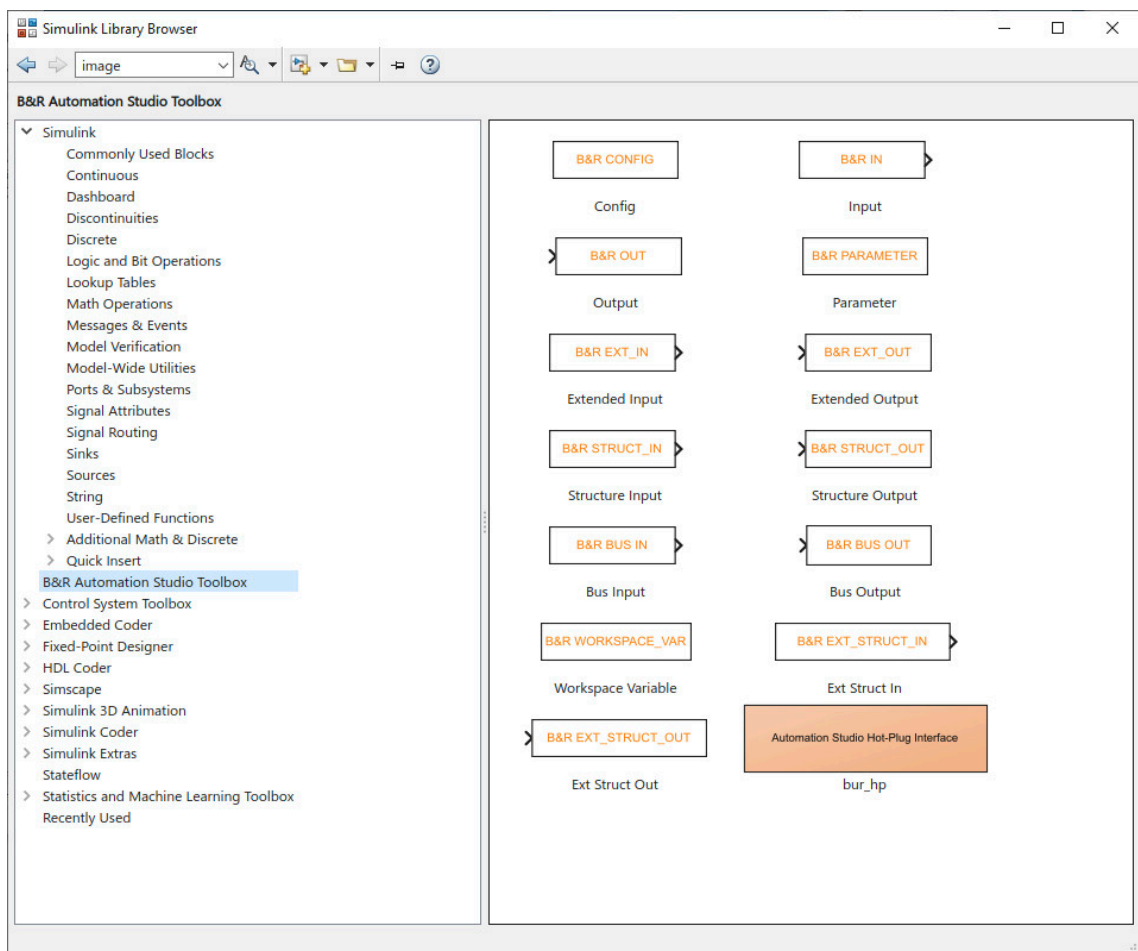


Figure 11: Automation Studio Target for Simulink library

### B&R Automation Studio Target for Simulink blocks

- The **B&R CONFIG block** is required to configure the Automation Studio connection and to select the coder and programming language (block cannot be generated).
- The **B&R IN block / B&R OUT block** creates a process variable in Automation Studio that can be linked to inputs or outputs.
- The **B&R PARAMETER block** creates a process variable in Automation Studio to make an internal parameter visible externally.
- The **B&R WORKSPACE\_VAR block** enables variables created in MATLAB/Simulink to be used in the Simulink model and generated as process variables in the Automation Studio project.
- The **B&R EXT\_IN block / B&R EXT\_OUT block** creates process variables in Automation Studio to adapt Simulink variables to the hardware inputs and outputs.
- The **B&R STRUCT\_IN block / B&R STRUCT\_OUT block** allows individual data types of existing Automation Studio structures to be used in the Simulink model. The structures are read from a specified .typ file.
- The **B&R EXT STRUCT Block** makes it possible to use entire existing Automation Studio structures in the Simulink model. The structures are read from a specified .typ file. The output/input of the block represents a Simulink.Bus. The \*.typ file can be used to initialize the bus signals with values.
- The **B&R BUS IN / B&R BUS OUT block** generates a corresponding structure in the Automation Studio .typ file from a Simulink bus variable as well as a corresponding process variable of the structure type.
- The **Automation Studio Hot-Plug Interface block** establishes a live ("Hot") connection, i.e. without code generation, to the controller/ARsim via the PVI protocol. Here, you have direct read/write access to PVI variables.

## 4.1 Installation and registration

Installation of Automation Studio Target for Simulink is started via the installation package (.exe). During installation, Automation Studio Target for Simulink must be licensed. The product can also be licensed later in MATLAB.

### 4.1.1 Installation

#### Requirements



#### IMPORTANT

A C/C++ compiler is required to compile Simulink models. This can be configured in MATLAB with command line command "mex -setup". For compatible compilers, see [de.mathworks.com/support/requirements/supported-compilers.html](https://de.mathworks.com/support/requirements/supported-compilers.html)

Example with Visual Studio C++ 2017 compiler:

#### Command Window

```
>> mex -setup
MEX configured to use 'Microsoft Visual C++ 2017 (C)' for C language compilation.

To choose a different language, select one from the following:
mex -setup C++
mex -setup FORTRAN
fx >> |
```

- C/C++ compiler (see info box above)
- MATLAB/Simulink
- Simulink Coder and/or Embedded Coder Toolbox (usually automatically included with the license when MATLAB/Simulink is installed)
- For integration in Automation Studio 3.0.90 or later
- .NET Framework V4.5 or later (latest version automatically installed if not found)
- Technology Guard software (latest version automatically installed if not found)



## Installation wizard

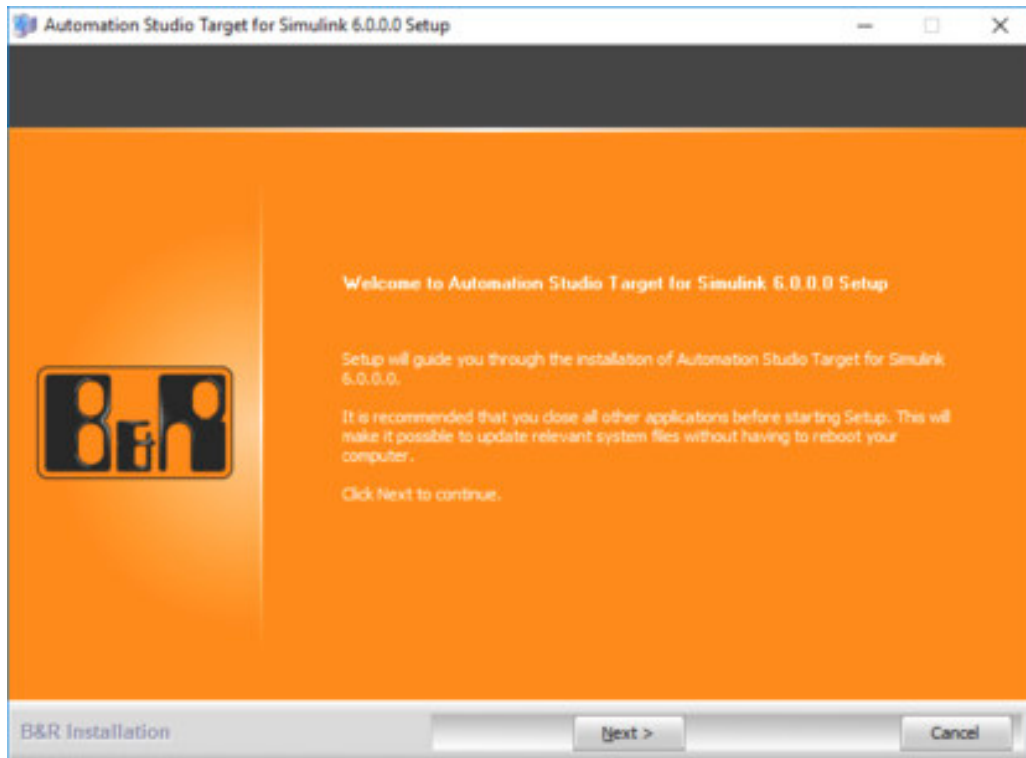


Figure 12: Automation Studio Target for Simulink - Welcome screen

Depending on which language is configured on the PC, the wizard is displayed in German or English.

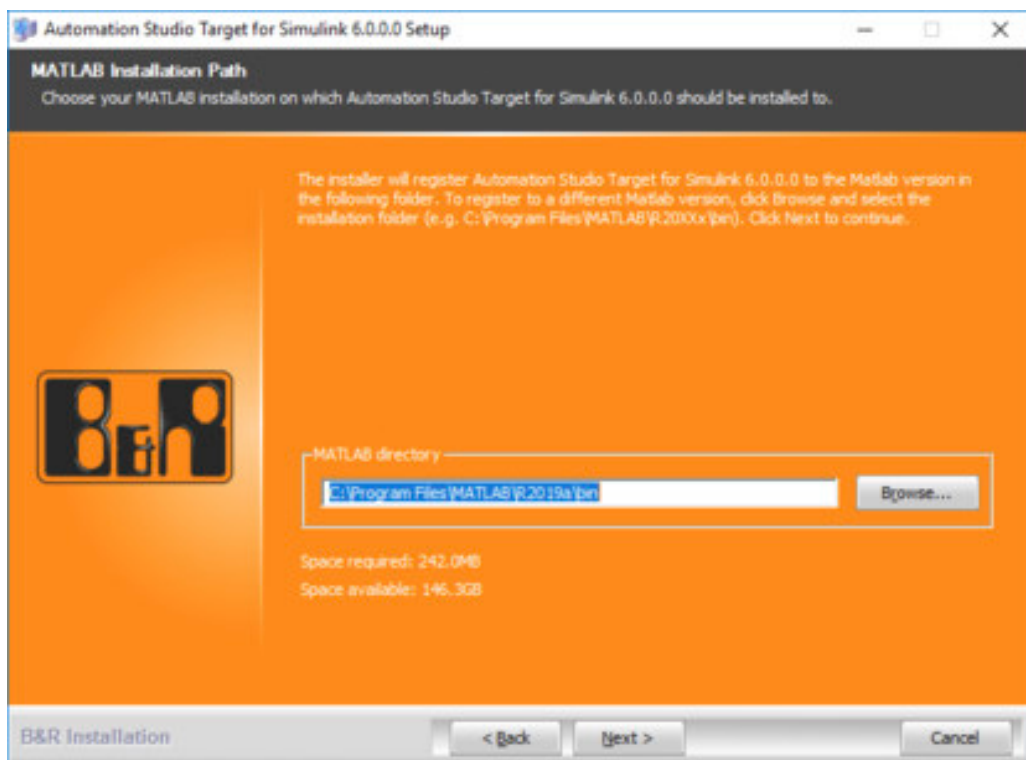


Figure 13: Specification of the MATLAB path where matlab.exe is located

Enter the MATLAB path to matlab.exe. MATLAB must be closed.

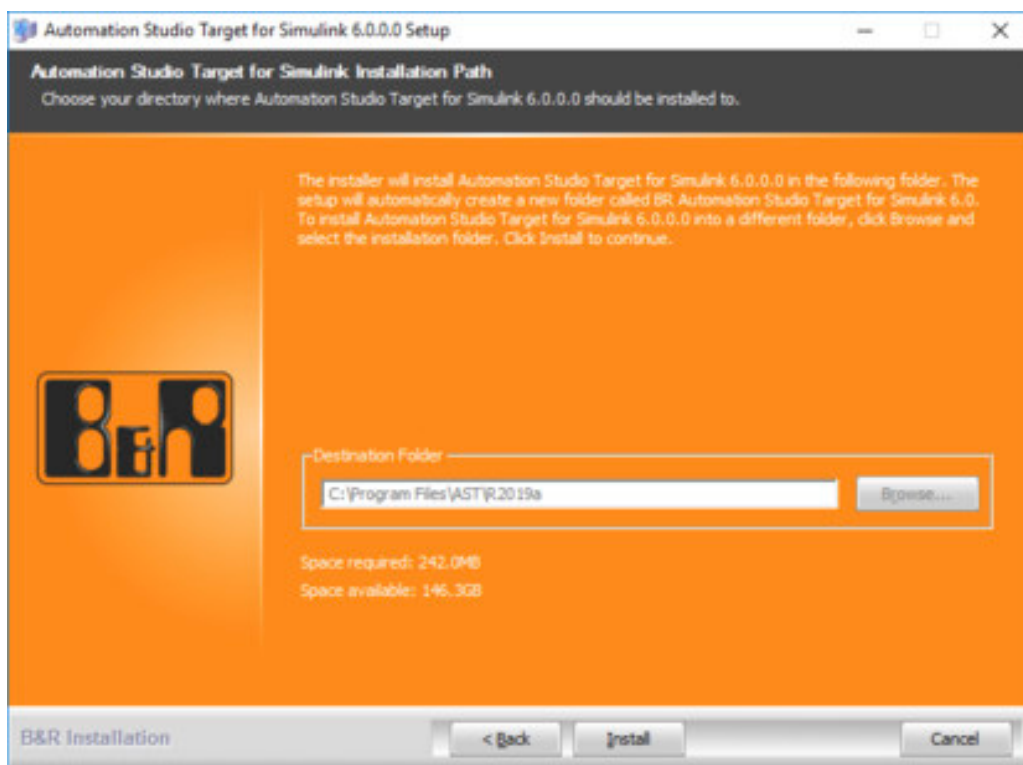


Figure 14: Specification of the Automation Studio Target for Simulink installation path

Enter the path for installing the B&R Automation Studio Target for Simulink files.

MATLAB will open in the background. Please wait until the licensing dialog box or a licensing message appears.

## 4.1.2 Uninstall procedure

The file for uninstalling is located in the installation directory of Automation Studio Target for Simulink or can be launched from **Control Panel / Programs and functions**.

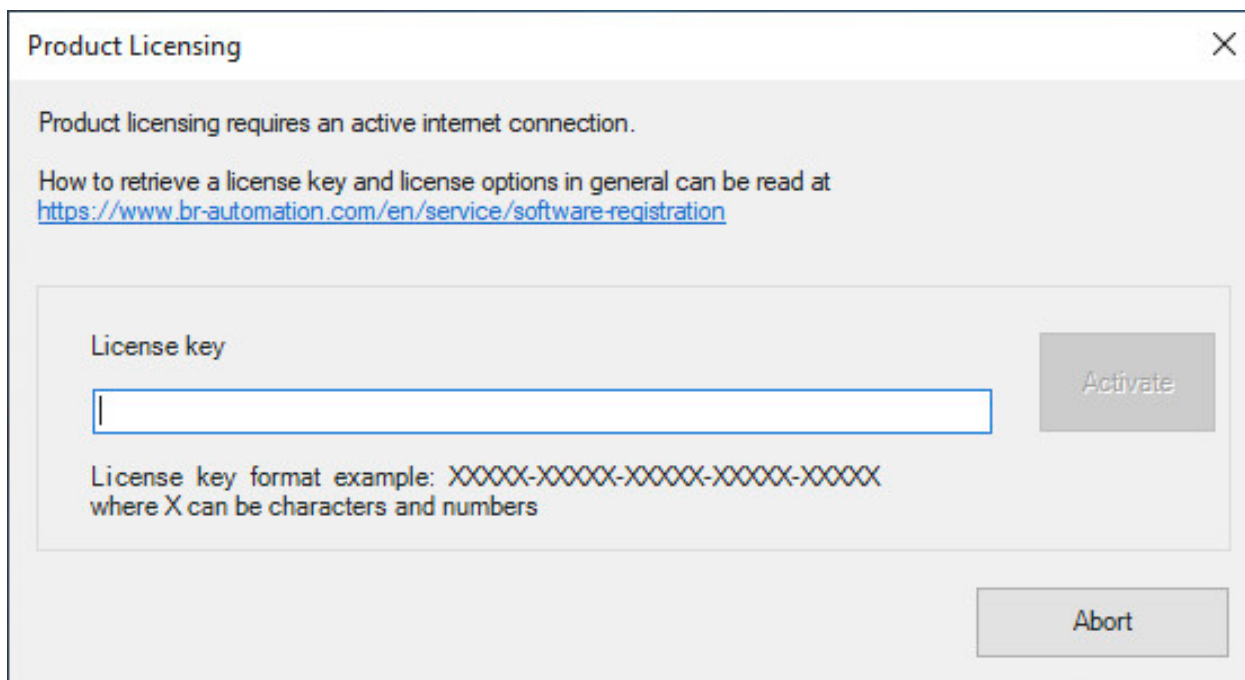
## 4.1.3 Licensing

### Licenses

| Automation Studio Target for Simulink                     | Technology Guarding / License name |
|-----------------------------------------------------------|------------------------------------|
| AS Target for Simulink - Single license (hardware dongle) | 1TGSIMTS.SNG.00-01                 |
| AS Target for Simulink - Site license                     | 1TGSIMTS.STE.00-01                 |
| AS Target for Simulink - Corporate license                | 1TGSIMTS.CRP.00-01                 |
| AS Target for Simulink - Education license                | 1TGSIMTS.STE.E0-01                 |
| AS Target for Simulink - Evaluation license               | 1TGSIMTS.TRL.00-01                 |
| AS Target for Simulink - Student license                  | 1TGSIMTS.STE.E1-01                 |
| AS Target for Simulink - Single-user software license     | 1TGSIMTS.SNG.00-02                 |

### Licensing procedure

Once Automation Studio Target for Simulink has been installed, it must be licensed. If Automation Studio Target for Simulink has not yet been licensed, the licensing dialog box is automatically displayed to enter a license key. If a Technology Guard dongle containing a license file exists, the license is activated automatically.



Depending on the selected license type, a Technology Guard for storing the license is needed (Technology Guarding) or a licensing key must be entered in the licensing dialog box. Evaluation licenses and student licenses can be requested directly on the B&R website.

**There are 2 options for licensing Automation Studio Target for Simulink.**

- Online activation with a valid license key
- Hardware dongle

#### **Online activation**

Enter the supplied B&R license key in the input field. Clicking button Activate will automatically license the B&R Automation Studio Target for Simulink version online over an existing Internet connection.

A 90-day evaluation license can also be requested. The link in the licensing dialog box directs to the [B&R website](https://www.br-automation.com/en/service/software-registration). When selecting an evaluation license, an email is sent to the specified address after the form has been filled out. This email contains a license key that can be activated in the licensing dialog box. The remaining evaluation period is shown each time Automation Studio Target for Simulink is started. After 90 calendar days, Automation Studio Target for Simulink must be licensed again.

#### **Hardware dongle**

Connect the supplied hardware dongle to an available USB slot and load the license onto the dongle via the Technology Guarding interface.

## **4.1.4 Upgrades**

The installer and latest version can be downloaded from the Downloads section of the B&R website.

Link: [B&R Downloads section](#)

## **4.2 Help documentation**

During installation of Automation Studio Target for Simulink, multiple help options are also installed. The **burhelp** folder, which contains the TM140 user's manual in German and English for Automation Studio Target for Simulink, can be found in the installation path. The auxiliary functions (commands) described below are also located here.

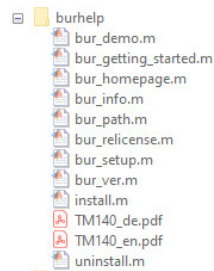


Figure 15: Automation Studio Target for Simulink burhelp folder

### List of help functions

- **bur\_ver**  
Returns the currently installed version of Automation Studio Target for Simulink for the current MATLAB version.
- **bur\_path**  
Specifies the path where Automation Studio Target for Simulink is installed.
- **bur\_setup**  
Allows you to switch between Automation Studio Target for Simulink versions starting with Automation Studio Target for Simulink version 5.3.0.
- **bur\_relicense**  
Opens a window to relicense Automation Studio Target for Simulink.
- **bur\_getting\_started**  
Opens the Automation Studio Target for Simulink help documentation.
- **bur\_demo (example\_number)**  
**bur\_demo** provides a list of all available B&R demo models to the command win-

```
>> bur_demo
Enter a number from 1-15 to open a B&R Demo example. A copy in the current path will be created, so you can edit it freely.
1 ... Example 1 - Basic examples
2 ... Example 2 - PID controller
3 ... Example 3 - Temperature model
4 ... Example 4 - Hydraulics application
5 ... Example 5 - Inverted pendulum
6 ... Example 6 - PID controller with Bus Objects
7 ... Example 7 - Simscape simple mechanical system
8 ... Example 8 - Data logger
9 ... Example 9 - Library Fub
10 ... Example 10 - Inverted Pendulum with Model Referencing
11 ... Example 11 - Inverted Pendulum with variant Subsystems
12 ... Example 12 - Fuzzy Logic PID Controller
13 ... Example 13 - Stewart Platform
14 ... Example 14 - Simulink IOs
15 ... Example 15 - PID with Data Dictionary
```

dow. >> |

The number in parentheses opens the respective demo Simulink model in a separate folder, e.g. `bur_demo(6)` opens "Example 6 - PID controller with bus objects". (write permissions must be available)

- **bur\_info**  
Opens a window with general information about Automation Studio Target for Simulink.
- **bur\_homepage**  
Opens the B&R website.

In addition to the help options mentioned above, there is also a section in Automation Help called **Simulation - Automation Studio Target for Simulink**.

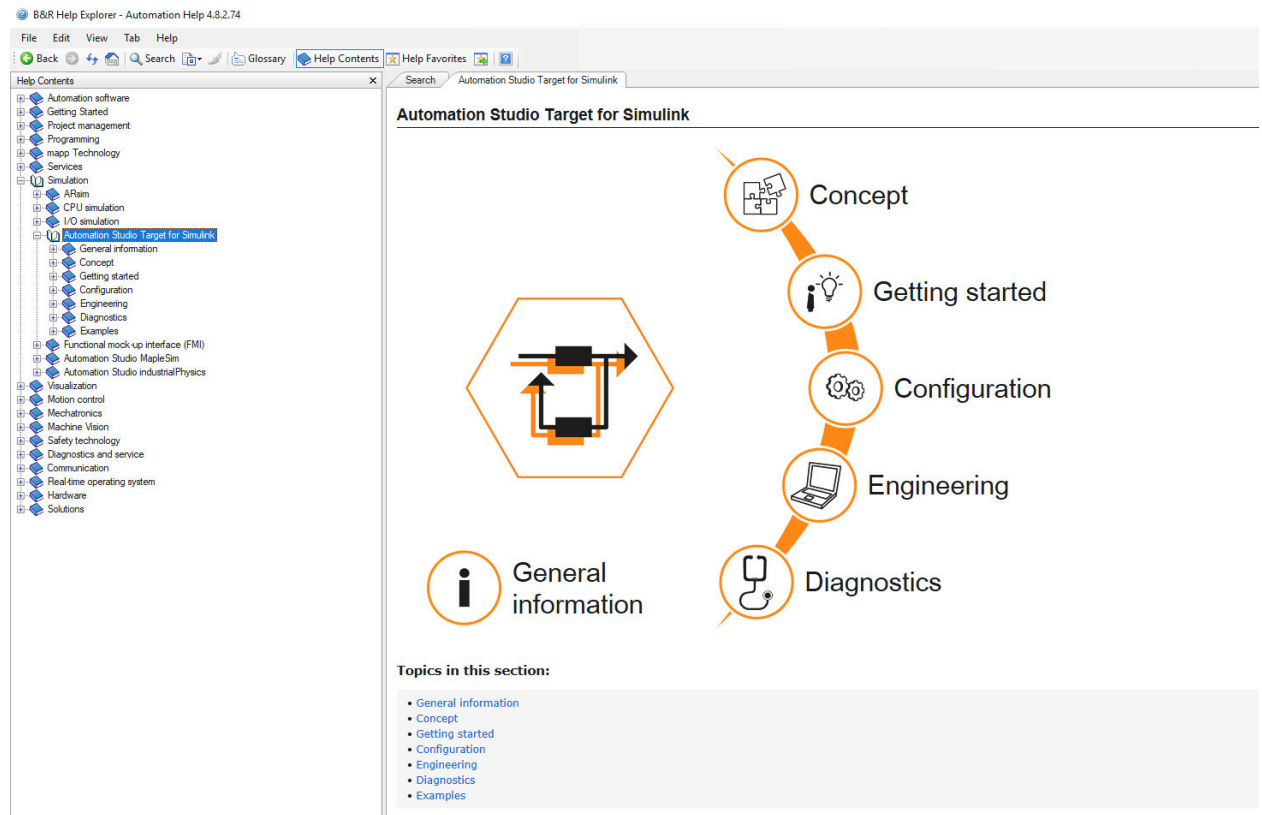


Figure 16: Automation Studio Target for Simulink in Automation Help

This essentially has the same content as the TM140, but appears in updated form with the respective Automation Studio version (not updated by an installation of Automation Studio Target for Simulink).

## 4.2.1 Demo models

During installation of Automation Studio Target for Simulink, demo models are also installed. These are located in the installation folder under **burdemos** and described in the Automation Studio Target for Simulink help documentation.

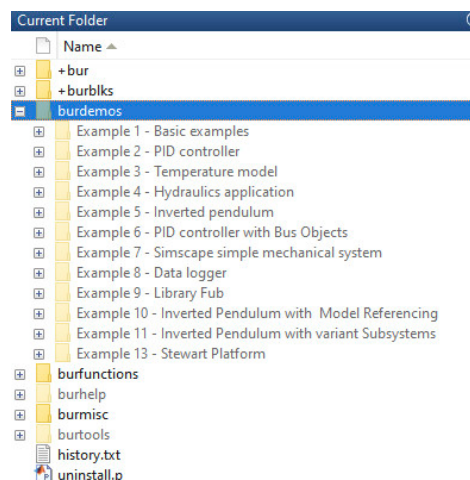


Figure 17: Demo models can be found in the installation folder under "burdemos".



The demo models can be opened by entering the demo model number in the command window.

## 5 My first project

This section covers creating a new model in Simulink, generating the code, importing it into an Automation Studio project, transferring it to Automation Runtime Simulation and testing it in Automation Studio.

### Exercise 1: My first project

The room temperature of a house is measured at two different points. Create a Simulink model called **WeatherStationSimple**, which calculates the average temperature and outputs it to an analog output. Only use the standard I/O blocks from Simulink and test the result using Simulink Simulation.



## 6 Principle of automatic code generation

The basic principle of **automatic code generation** from MATLAB/Simulink into Automation Studio consists of three elements. Creating a Simulink model in MATLAB/SIMULINK, integrating the generated program code in Automation Studio and running the project on the target system.

The model created in Simulink is automatically converted to the C/C++ programming language by Simulink Coder or Embedded Coder (optional). The Automation Studio Target for Simulink interface is automatically adapted and integrated with Automation Studio.

The project can then be transferred to the target system. The target system can now communicate not only with Automation Studio but also with the model in Simulink.

This allows simple transfer of complex and sophisticated Simulink models to the target system.

### Five steps to executable program code

- Create a Simulink model
- Add a B&R config block to the Simulink model.
- Create an Automation Studio project
- Configure the Automation Studio Target for Simulink parameters
- Start automatic code generation

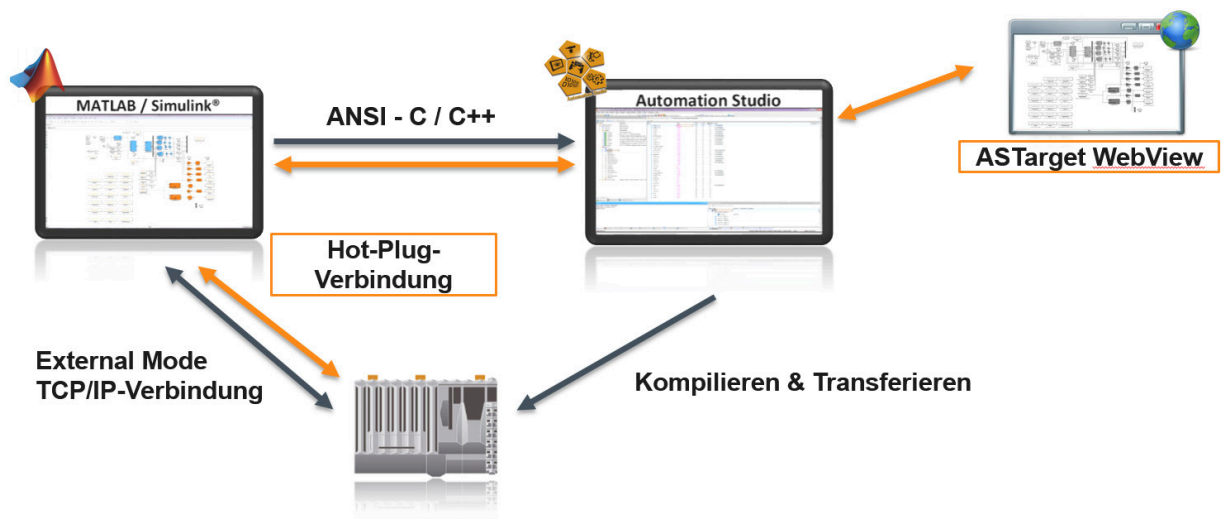


Figure 18: Workflow for Automation Studio Target for Simulink

### Exercise 2: Code generation

After successful simulation, the code for the preceding example in exercise 1 should be generated in an Automation Studio project.



- 1) Create a new Automation Studio project. Hardware: e.g. PC - Automation Runtime Simulation
- 2) Add a B&R config block to the Simulink model from the **My first project** exercise.



- 3) Apart from that, only use standard Simulink I/O blocks.
- 4) Configure the B&R Config block to generate standard Simulink I/O blocks.
- 5) Configure the B&R config block so a task is created in Automation Studio.
- 6) Start code generation. What information is output during code generation? What files are created in MATLAB in the background?
- 7) Transfer the Automation Studio project to the Automation Runtime Simulation and check the functionality of the generated program code in monitor mode.

# 7 Interface to Automation Studio - B&R config block

The **B&R config block** is the interface between the Simulink model and Automation Studio. Each Simulink model that should be integrated into an Automation Studio project via automatic code generation must include an instance of the B&R config block. The block is used to set B&R-specific parameters as well as important Simulink-specific model parameters and to select the coder and language to be generated.

The B&R config block user interface consists of the five tabs **Model configuration**, **Automation Studio settings**, **Advanced settings**, **Additional files** and **Information**. The primary settings are explained in the following chapters. For additional information, see the Automation Studio Target for Simulink help documentation.



Only one instance of the B&R config block is allowed and necessary per Simulink model. The B&R config block must always be created in the highest hierarchy level of the Simulink model.



Automation Studio Target for Simulink help documentation

- Chapter 2.1 B&R config block
- Chapter 3 Configuration settings

## 7.1 Basic settings

The information and parameters necessary for automatic code generation are set in the basic settings for the B&R config block. The basic settings include tabs **Model configuration** and **Automation Studio settings**.

### Model configuration

Tab **Model configuration** can be used to set important Simulink-specific model parameters and the code generation mode.

Four code generation modes are available for a Simulink model.

- Simulink Coder C
- Simulink Coder C++
- Embedded Coder C
- Embedded Coder C++

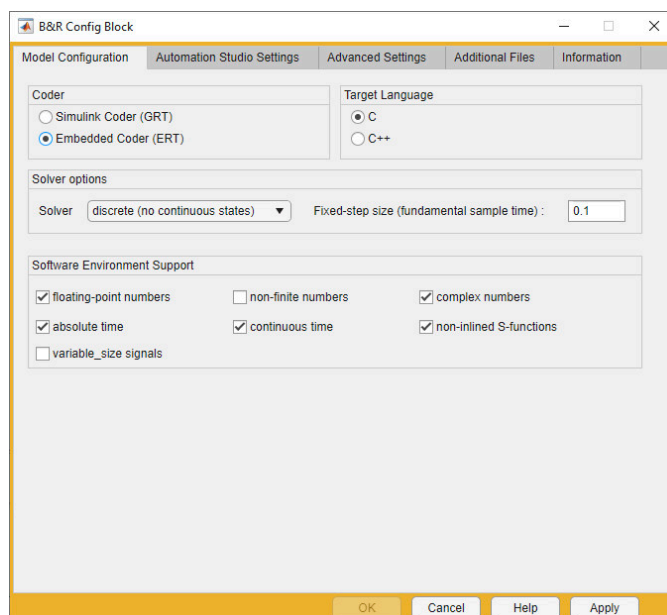


Figure 19: B&R config block user interface - Model configuration

In the model workspace, a separate configuration is created in the background for each code generation mode. It is possible to switch between modes by first using the radio buttons for selection and then clicking on button "OK" or "Apply".



Selection of the Embedded Coder as a configuration option only appears if it is available on the system.

### Automation Studio settings

Important information for automatic integration of the generated code into an Automation Studio project can be set in the **Automation Studio settings**. If the generated program code should be Automation Studio compatible but not directly integrated, it can be saved as a **ZIP package**. This can be enabled with the **Create ZIP package** option.

In the further course of action, a decision must be made between **task** and **function block** generation. When generating a function block, the generated program code is encapsulated in a function block and a library is created that includes this function block. Task generation, however, uses the generated program code to create a stand-alone program unit in Automation Studio. In both cases, the program code is automatically added to the Logical View in Automation Studio.



The generated code requires B&R standard libraries **brsystem** and **sys\_lib**. These are automatically added to the Automation Studio project during task or function block generation. When creating a ZIP package, these libraries must be manually added to the Automation Studio project. Additionally, it is necessary to make the settings defined in `task_properties.txt` / `lib_properties.txt` (a file that is also generated).

The screenshot shows the 'B&R Config Block' dialog box with the 'Automation Studio Settings' tab selected. The 'Code Generation Path' section has a text field for 'Automation Studio Project Path / Zip Path' and a 'Browse' button. Below it is a checkbox for 'Create Zip Package'. The 'Basic Parameters' section has radio buttons for 'Task' (selected) and 'Function Block', with corresponding text fields for 'Task Name' and 'Function Block Name'. There are also checkboxes for 'Add Task to Package' and 'Description'. The 'Library - Function Block Options' section has checkboxes for 'Legacy Mode' and 'Create B&R Library guidelines compliant Function Block'. The 'Configuration Parameters' section has checkboxes for 'Add Task to Hardware' and 'Automatic Transfer', and a 'Configuration Name' dropdown. The 'PLC Configuration' section has a 'Task Class' dropdown set to 'Cyclic #1', a 'Change Cyclic Time' checkbox, and input fields for 'Duration [s]', 'Tolerance [s]', and 'System Tick [s]'. At the bottom are 'OK', 'Cancel', 'Help', and 'Apply' buttons.

Figure 20: User interface B&R config block - Automation Studio settings

You must also specify the path where the Automation Studio project is located or the ZIP package should be created. The path can be entered using the **Automation Studio project path / ZIP path** field. The name of the task to be created, the library and the function block must also be specified (**Task name / Library name / Function block name**).



The name of the model and the task or library and function block are not permitted to be identical.

## 7.2 Automatic transfer

Setting **Automatic transfer** is located in tab **Automation Studio settings**. If enabled, the generated program is compiled and transferred to the target system after automatic code generation.

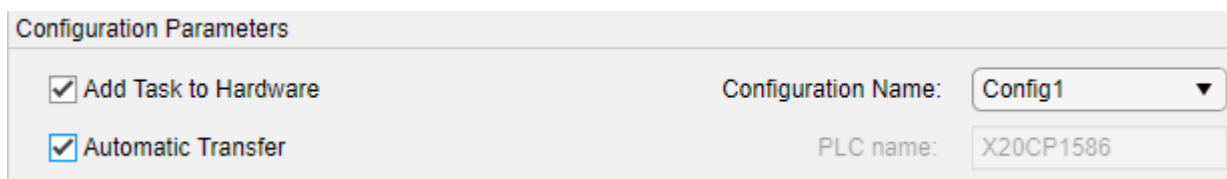


Figure 21: Enabling automatic transfer in the B&amp;R config block

**Ethernet connection**

Automation Help: Programming \ Build & Transfer \ Establishing a connection to the target system \ Ethernet connection

## 7.3 External mode

The Simulink **external mode** configuration setting enables communication at runtime between the Simulink model and the code-generated model running on the target system.

External mode establishes one communication service each on the system on which MATLAB is installed (PC) and the target system (controller or PC). These two services on the respective systems establish a communication channel between the Simulink engine and the generated code provided on the target system. The communication service isolates the model process on the target system from the code and transport layer. The tasks of the transport layer are to format, transmit and receive data packets. The communication service on the host computer receives the data packets through the transport layer and updates the Simulink model display. The diagram shows the connection that the communication service establishes in external mode between Simulink on the host computer and the provided code on the target system.

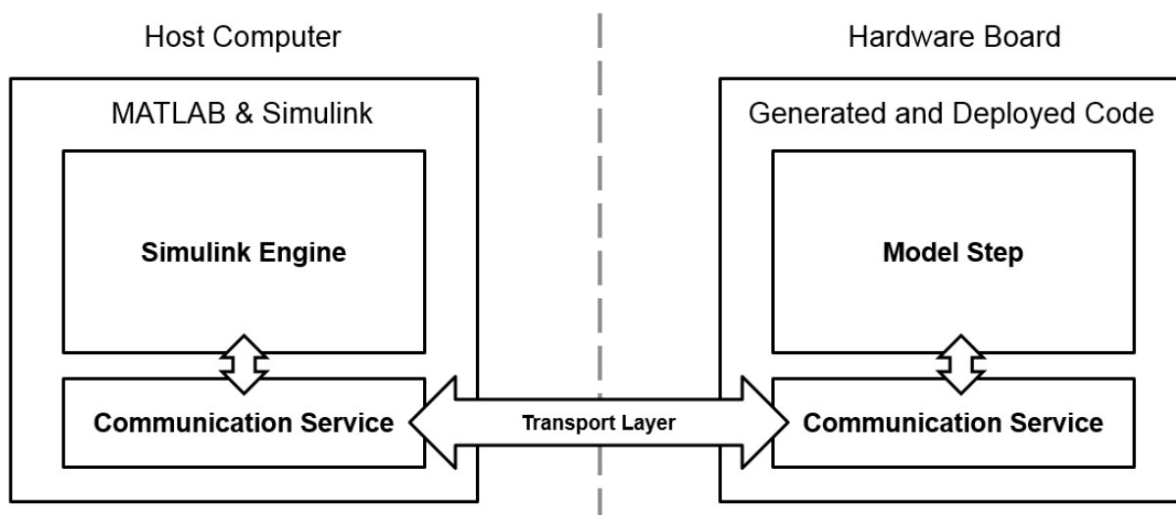


Figure 22: External mode communication scheme MATLAB/Simulink

### Configuring external mode

To use external mode, the feature must be enabled in the B&R config block in tab **Advanced settings**. In addition, three settings must be made:

- **IP address:** IP address of the target system. The "localhost" IP address 127.0.0.1 is entered as the default value (e.g. when using ARsim).
- **Port:** External mode uses the port for communication with Automation Studio. The port number is an integer value between 1024 and 65535. The MATLAB default value is port number 17725.
- **Buffer size:** Buffer size that is preallocated on the target system. Default value: 1000000 bytes (1 MB)

B&R libraries **AsArLog** and **AsTCP** are necessary for external mode in Automation Studio. They are automatically added to the Automation Studio project when a task or function block is created. If a ZIP package is created, these libraries must be added manually in the Automation Studio project.



Figure 23: External mode settings in the B&R config block

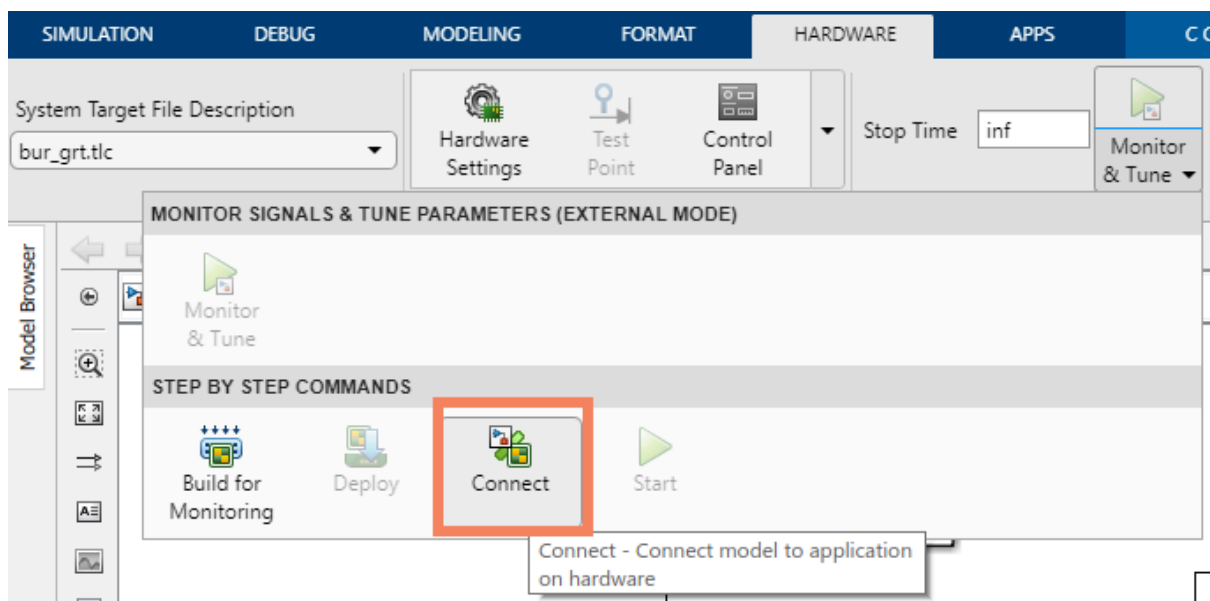


Enabling external mode generates additional code during model code generation that affects performance. For additional information, see the documentation for [External mode](#) from MATLAB.

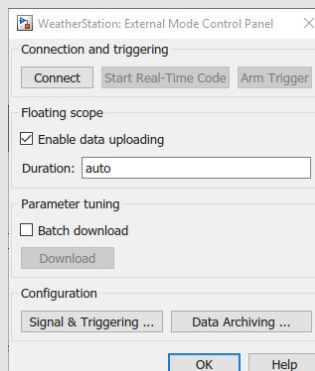
**Link:** <https://de.mathworks.com/help/rtw/ug/set-up-and-use-hosttarget-communication-channel.html>

### Starting external mode

- 1) Enable and specify external mode features in the B&R Config block.
- 2) Generate code and start the target system.
- 3) Configure the Simulink model for external mode and start the external mode connection.



External mode can also be started and stopped via the **External Mode Control Panel** with the connect/disconnect button.



### Manipulating variables on the target system from Simulink

While communication between the Simulink model and target system is taking place, process variables can be observed and manipulated in the Simulink model.

#### Displaying process variables in the Simulink model

To check current values in the Simulink model at runtime, blocks from Simulink library "Sinks" can be used (e.g. "Scope" or "Display").

#### Manipulating process variables in the Simulink model

There are two ways to manipulate parameters from Simulink. The first is modifying the values of Simulink blocks like **Gain** or **Constant** directly during the external mode connection. The other is to use the **External Mode Control Panel**. The External Mode Control Panel allows values of workspace variables to be changed locally in Simulink and then downloaded to the target system using command **Batch download**. This can be used with the B&R Parameter block or B&R Workspace Variable block, for example. Only parameters that are located in the MATLAB workspace require the batch download. (B&R input/output parameters cannot be changed because they are not in the workspace, for example) The External Mode Control Panel is located under Code > External Mode Control Panel > Batch download > Download.

## 7.4 Cycle time and task class

If a program in Automation Studio is added to the Logical View via the toolbox, then it is automatically included in the software configuration for the active configuration.

In the **Automation Studio settings** tab, the B&R config block provides the possibility to automatically assign the task to be generated to a configuration using the **Add task to hardware** option in the Automation Studio project. Selecting this option enables options **Target task class** and **Change cyclic task class**. The "Target task class" option makes it possible to assign the task to a certain task class. Task class #1 is defined as the standard task class.

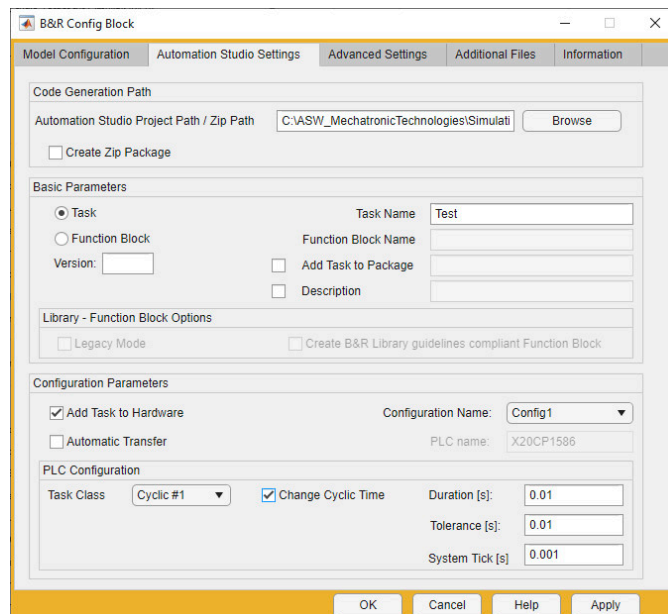
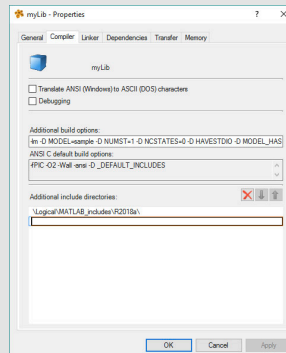


Figure 24: Enabling options "Add task to hardware", "Target task class" and "Change cyclic task class"



If the generated task is automatically assigned to the correct configuration and task class, the additional compiler switches and additional include directories are also specified. These entries are always stored again in the generated code for manual integration in an additional text file.



In addition to the task class assignment, changes can also be made to the selected task class by activating the **Change cyclic task class** option.

#### Duration [s]:

Duration of the **cycle time**. Configurable range: 400  $\mu$ s to 60 s. The value must be a multiple of the system tick. All software objects for this resource are processed exactly once within this cycle time.

#### Tolerance [s]:

Maximum allowed cycle time violation. This value must be a multiple of the cycle time. This allows the cycle time to be exceeded by a predefined amount.

#### System tick [s]:

Influences the timing characteristics of the system. A system tick is the time base for the clock generator used to derive the cycle time (also for the task classes). By default, the system tick is set to 0.001 seconds. The system tick is adjusted in the CPU configuration in the "Timing" subsection.



For more information, see Automation Help:

Real-time operating system \ Method of operation \ Runtime performance \ Task classes

Project management \ Configuration View \ Properties of the objects in the Configuration View

Programming \ Editors \ Configuration editors \ Hardware configuration \ CPU configuration \ SG4 \ CPU properties - Timing

Programming \ Editors \ Configuration editors \ Hardware configuration \ CPU configuration \ SG4 \ CPU properties - Resources

## 7.5 Dependence on additional files

The B&R config block offers two possibilities in the **Additional files** tab.

- Adding additional source files or other files to the generated task or function block.
- Specification of the type of files whose structures are read and then available in the Simulink model. For more information, see chapters [10.3 "B&R struct block"](#) and [B&R extended struct block](#).

## 7.6 Additional functionality

In addition to the options described above, the B&R config block also offers other options in the "Advanced settings" tab. Some of the important ones are described below.

- **Create Simulink I/Os/ Simulink.Parameter / Simulink.Signal**  
Creates all I/O variables defined by standard Simulink blocks, Simulink.Parameter variables or Simulink.Signal variables that are defined in the base workspace or data dictionary as local/global variables in the generated code and creates them in the VAR file.
- **Enable pre/post processing**  
Before/After the code generation has been completed, an M-file can be called up for a pre/post processing routine. The file must be located in a MATLAB path.
- **Create global var files**  
Creates a global\_modelName.var and global\_modelName.typ file with all IO variables defined globally in the Simulink model.
- **Simulink web interface**  
Creates a web service on the target system for accessing the Simulink model via a web browser. Intended for diagnostic purposes.



For additional information, see the Automation Studio Target for Simulink help documentation.  
Chapter 2.1 B&R config block  
Chapter 3 Configuration settings



## 8 Variables, constants and arrays

Programming in Automation Studio is not done by accessing fixed memory addresses, but instead via symbolic elements that have names. These elements are called **process variables**. Process variables compatible with Automation Studio are created with B&R Simulink blocks that are added to the Simulink library when Automation Studio Target for Simulink is installed.

The following sections explain how to create **variables**, **constants** and **arrays**.

### Declaration

In Automation Studio, variables, constants and arrays are only created in files with the \*.var file extension. Accordingly, when a program with newly generated code is added, a variable declaration with the name of the program is created so it is available locally.

### Initialization

In Automation Studio, variables, constants and arrays can be initialized directly in the declaration editors. As a result, an initialization value can be assigned in Simulink, which is then automatically applied and displayed in the declaration editor.

### Scope

The scope of declared process variables depends on the position of the declaration file in the Logical View. For each variable, constant or array, you can define whether it is available globally or locally. When available globally, a global variable declaration is created.

### Basic data types

Basic data types form the basis for all other derived data types. Basic data types determine the value range of a variable in addition to how much space it needs in memory. Data types also establish whether values are signed or unsigned, whether they include decimal places, text or even dates or times.

The following list shows all basic data types standardized according to IEC 61131-3 that are supported by Automation Studio Target for Simulink and their counterparts in Simulink.

|                     | Automation Studio | Simulink | Range of values                  |
|---------------------|-------------------|----------|----------------------------------|
| Binary / Bit string | BOOL              | Boolean  | FALSE, TRUE                      |
| Signed integers     | SINT              | int8     | -128 ... 127                     |
|                     | INT               | Int16    | -32768 ... 32767                 |
|                     | DINT              | int32    | -2,147,483,648 ... 2,147,483,647 |
| Unsigned integers   | USINT             | uint8    | 0 ... 255                        |
|                     | UINT              | uint16   | 0 ... 65535                      |
|                     | UDINT             | uint32   | 0 ... 4,294,967,295              |
| Floating point      | REAL              | Single   | -3.4E+38 ... 3.4E+38             |
|                     | LREAL             | double   | -1.7E+308 ... 1.7E+308           |

Table 1: Overview of supported IEC data types

### 8.1 Code generation

After successful simulation of the Simulink model, the model should be generated in an Automation Studio project using the B&R Config block. To do this, the B&R Config block is added to the model, and the project path and task name are set in the block.

Afterwards, the variables for the inputs and output should be created manually in the project so that the generated code can be tested on ARsim.

### Exercise 3: Code generation with B&R I/Os

The standard Simulink I/Os from the previous example in exercise 2 should now be replaced by B&R blocks and code should be generated again.



#### Declaration

VAR

```
aiSummand1: REAL; (*erster Temperatursensor*)
aiSummand2: REAL; (*zweiter Temperatursensor*)
aoAvgTemp: REAL; (*Durchschnittstemperatur*)
```

END\_VAR

Table 2: Variable declaration of process variables created with B&R blocks

- 1) Use the model from the previous exercise (exercise 2) and add the appropriate B&R Input blocks and a B&R Output block. Check the model with the simulation.
- 2) Start the code generation.
- 3) Check the variables in the VAR file and check the functionality of the generated program code in the Watch window.

#### Exercise 4: Change the task class and cycle time

Use the example from exercise 3 **Code generation** to generate the task in a specific task class. The cycle time for this task class should also be adapted.

- 1) Change the task class.
- 2) Modify the selected task class. Change the cycle time to 0.01 s.
- 3) Generate the model in an Automation Studio project and check the modifications.
- 4) Determine what happens if the cycle time of the task class does not match the step size of the Simulink model.



The cycle time of the task class must match the step size of the Simulink model.

## 8.2 B&R input block

The B&R input block creates a process variable per IEC in Automation Studio that can be linked to inputs. The interface allows you to specify all the information that is also possible in the .var file in Automation Studio.

Figure 25: B&R input block - User interface

**Variable name:** Name of the process variable in Automation Studio.

**Variable description:** Comment entry for the description of process variables.

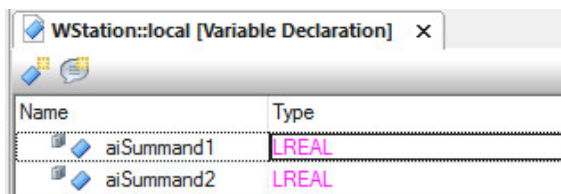
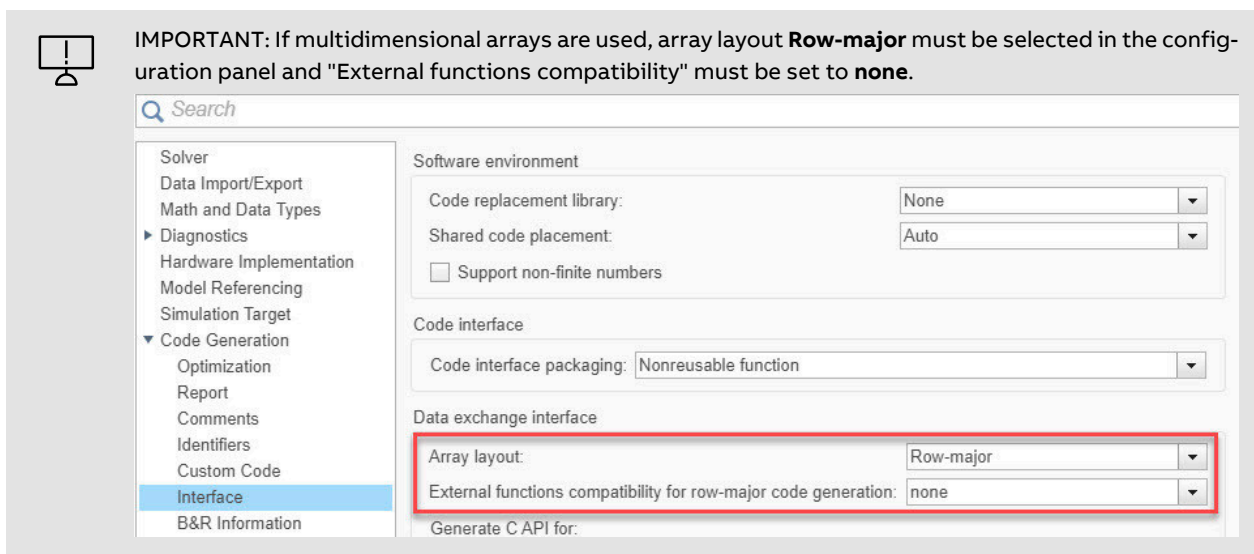
**Variable scope:** Availability and visibility of the process variable within the entire project. Values: "Global" or "Local".

**Variable data type:** Data type of the process variable per IEC 61131-3.

**Memory:** Decides whether the process variable is battery-backed (value retained after the controller is switched off). Values: "Retain" for battery-backed or "Standard" for not battery-backed.

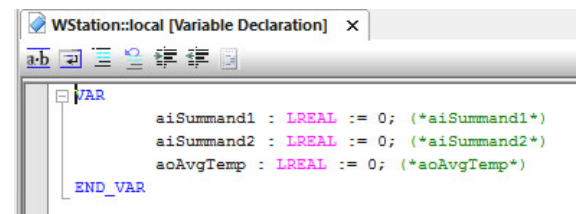
**Initial value:** Initial value of the process variable. This value is used for all fields of an array. To initialize an n-dimensional array with different initial values, square brackets must be written in the field. (For example: [1,2,3;4,5,6] for a 2-dimensional array with the arrangement [2,3], i.e. 2 rows and 3 columns)

**Array size:** If the value is 1, a process variable is created as a variable. If the value is greater than 1, a process variable is created as a one-dimensional array. To create an n-dimensional array, square brackets must be written in the field (e.g. [3,5] for a 2-dimensional array with the arrangement 3 rows and 5 columns). It is also possible to write variables instead of numeric values into the field, e.g. [rowSize,colSize] if rowSize and colSize are declared accordingly in the workspace.



| Name       | Type  |
|------------|-------|
| aiSummand1 | LREAL |
| aiSummand2 | LREAL |

Figure 26: Displaying a declaration as a table



```
VAR
    aiSummand1 : LREAL := 0; (*aiSummand1*)
    aiSummand2 : LREAL := 0; (*aiSummand2*)
    aoAvgTemp : LREAL := 0; (*aoAvgTemp*)
END_VAR
```

Figure 27: Displaying a declaration as text

### 8.3 B&R output block

The B&R output block creates a process variable per IEC in Automation Studio that can be linked to outputs.

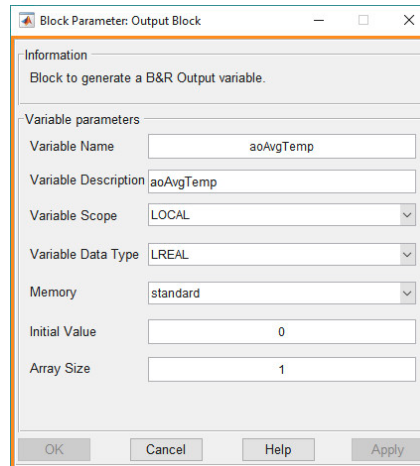


Figure 28: B&R output block - User interface

The same setting options are available as for the B&R input block.

### 8.4 Creating internal variables and constants

Variables that are used internally in the model can be made visible in Automation Studio. These variables can also be defined as constants using the **B&R parameter block**. The value cannot change at runtime. Constants are used in the program code, for example, as limit values.

There is also the option "**Create Simulink.Parameter/Signal**" in the **B&R Config block**, which generates code for variables that have already been created in MATLAB. It is important to ensure that "**Storage class**" is set to "**ImportedExternal**".

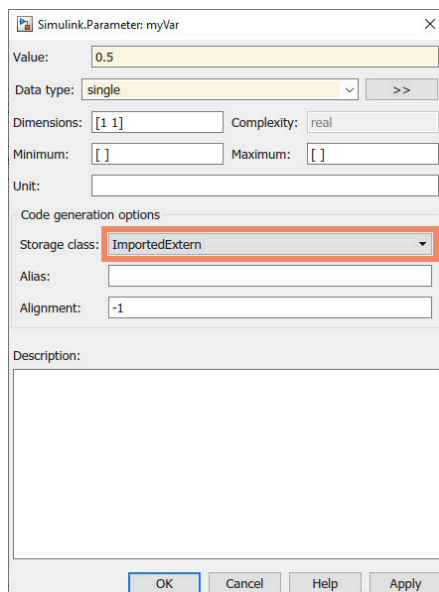


Figure 29: Setting the storage class for the code generation on ImportedExtern

#### Declaration

When a program with newly generated code is added, a variable declaration with the name of the variable is created so it is available locally.

For each constant, it is also possible to define individually that it is available globally. In this case, a global variable declaration is created.

The B&R parameter block can be used to create constants in Automation Studio.

#### Exercise 5: Make internal variables visible

The value of the gain block should remain variable. Accordingly, the value of the gain block should be visible in Automation Studio as a process variable.

- 1) Add a B&R parameter block from the **Create process variables** exercise to the model.
- 2) Start the code generation.
- 3) Try to change the value of the variable using the External Mode Control Panel
- 4) In the second step, it was decided that the variable for the gain block would be created as a constant. Change the settings in the Simulink model so the variable is created as a constant in Automation Studio.
- 5) Now, remove the B&R parameter block and create a Simulink parameter variable. Repeat step 2.



### 8.4.1 B&R parameter block

The B&R parameter block offers the option of making internal variables of the Simulink model that are not inputs or outputs visible in Automation Studio as IEC process variables in the .var file.

A special feature of the B&R input/output block is that the process variable can be defined as a constant.

Figure 30: B&R parameter block - User interface

**Variable name:** Name of the process variable in Automation Studio.

**Variable description:** Comment entry for the description of process variables.

**Variable scope:** Availability and visibility of the process variable within the entire project. Values: "Global" or "Local".

**Variable data type:** Data type of the process variable per IEC 61131-3.

**Memory:** Decides whether the process variable is battery-backed (value retained after the controller is switched off). Values: "Retain" for battery-backed, "Standard" for not battery-backed, "Constant" for defining a constant.

**Initial value:** Initial value of the process variable. This value is used for all fields of an array. To initialize an n-dimensional array with different initial values, square brackets must be written in the field. (For example: [1,2,3;4,5,6] for a 2-dimensional array with the arrangement [2,3], i.e. 2 rows and 3 columns)

**Array size:** If the value is 1, a process variable is created as a variable. If the value is greater than 1, a process variable is created as a one-dimensional array. To create an n-dimensional array, square brackets must be written in the field (e.g. [3,5] for a 2-dimensional array with 3 rows and 5 columns). It is also possible to write variables instead of numeric values into the field, e.g. [rowSize,colSize] if rowSize and colSize are declared accordingly in the workspace.

## 8.5 Creating arrays

In contrast to basic data type variables, values with the same data type are combined in arrays. The individual elements can be addressed with the array name and an index.

The value of the array index is not allowed to exceed the array size. The size of an array is defined by the variable declaration. In the program, the index can be a fixed value, a variable, a constant or an enumerated element.

### Declaring and using arrays

When an array is declared, it must be given a data type and a dimension. In most programming languages, the smallest index of an array is set to 0 (MATLAB is an exception and starts with index 1). For this reason, take note that an array with 10 elements has a maximum index of 9 (apart from MATLAB).

|  |                     |                                                                     |
|--|---------------------|---------------------------------------------------------------------|
|  | <b>Declaration</b>  | <pre>VAR   aPressure : ARRAY[0..9] OF INT := [10(0)]; END_VAR</pre> |
|  | <b>Program code</b> | <pre>(*Assigning value 123 to index 0*) aPressure[0] := 123;</pre>  |

Table 3: Declaring an array of 10 elements in Automation Studio, starting index = 0

### Declaring an array using constants

Since using fixed numeric values in declarations and the program code itself (so-called magic numbers) usually leads to programming that is unmanageable and difficult to maintain, it is a much better idea to use numeric constants. The upper and lower indexes of an array can be defined using these constants. These constants can then be used in the program code to limit the changing array index.

|  |                     |                                                                                                                                        |
|--|---------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|  | <b>Declaration</b>  | <pre>VAR CONSTANT   MAX_INDEX : USINT := 9; END_VAR VAR   aPressure : ARRAY[0..MAX_INDEX] OF INT ;   index : USINT := 0; END_VAR</pre> |
|  | <b>Program code</b> | <pre>IF index &gt; MAX_INDEX THEN   index := MAX_INDEX; END_IF aPressure[index] := 75;</pre>                                           |

Table 4: Declaring an array in Automation Studio using a constant



In C/C++, the first element of an array is addressed with 0; in MATLAB, arrays always start with 1. Consequently, an array with 5 elements in C/C++ has an index from 0 to 4. In MATLAB, however, the array has an index from 1 to 5.

### 8.5.1 Creating one-dimensional arrays

The **B&R input/output block** and the **B&R parameter block** allow process variables to be created as one-dimensional arrays in Automation Studio. The value in option **Array size** is all that needs to be adjusted.

If the array size is set to 5, a process variable is created in Automation Studio as a one-dimensional array with 5 elements that can be addressed using the index from 0 to 4.



There are various ways to initialize the elements of the one-dimensional field. In "Initial value", it is possible to enter either a single number (all fields are then initialized with this value) or square brackets and a comma (the individual field values can then be determined, e.g. [1,2,3,4] for a field with 4 elements).

Exercise 6: Create one-dimensional arrays

House temperature control gets an update. The temperatures of the last three days should be stored in order to calculate the average temperature of these days. Modify the example from exercise 5 **Make internal variables visible** to meet the new requirements.

- Change the array size of all B&R blocks to the corresponding days.
- Start the code generation.
- Try to change the values of the B&R parameter variable again using the External Mode Control Panel.



8.5.2 Creating multidimensional arrays

Arrays can also be composed of several dimensions. The declaration and usage in this case can look something like this:

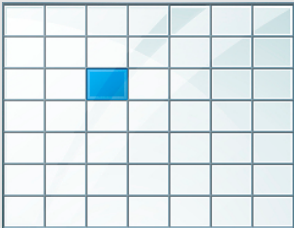
|                                                                                                      |                                                                  |                                                                                      |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|  <b>Declaration</b> | <pre>VAR     Array2Dim : ARRAY [0..6,0..6] OF INT; END_VAR</pre> |                                                                                      |
| <b>Program code</b>                                                                                  | <pre>(*Zählweise mit 0 beginnend*) Array2Dim[2,2] := 11;</pre>   |  |

Figure 31: Accessing the value in Column 3, Row 3

Table 5: Declaring and accessing a 7x7 two-dimensional array

The **B&R input/output block** and the **B&R parameter block** allow process variables to be created as multidimensional arrays in Automation Studio. The value in option **Array size** is all that needs to be adjusted. If the array size is set to [2,3], a process variable is created in Automation Studio as a 2-dimensional field with 6 elements.



There are various ways to initialize the elements of the multidimensional field. In the item "Initial value", it is possible to enter either a single number (all fields are then initialized with this value) or square brackets, a comma and a semicolon (the individual field values can then be determined, e.g. [1,2,3,4,5,6] for a [2,3] field with 6 elements).



By default, MATLAB generates one-dimensional arrays in C from multidimensional arrays in Simulink (the Embedded Coder provides advanced settings that preserve multidimensionalities in the generated code). The **B&R input/output block** and the **B&R parameter block** take this conversion into account, but still create the arrays as multidimensional in the VAR file in order to retain the desired structure. This conversion was not implemented for the **B&R workspace block**.

8.5.2.1 B&R workspace variable block

The B&R workspace block makes it possible to create existing variables created on a workspace as process variables in Automation Studio. A common use case is to create fields with different initial values as process variables. The block supports the following:



- Variables of data type "Double"
- One-dimensional and multidimensional arrays of data type "Double"
- Structures
- Simulink.Parameter
- Mpt.Parameter

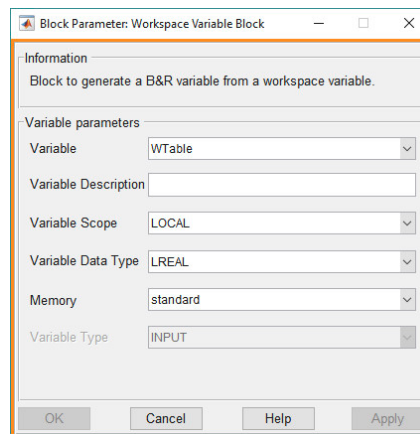


Figure 32: B&amp;R workspace variable block - User interface

### 8.5.2.2 Creating 2-dimensional fields

Creating multidimensional fields can be done in several ways. In this example, the input and output will be created as 2-dimensional fields.

#### Exercise 7: Create 2-dimensional fields

Using the model from the previous exercise, create a 2-dimensional array using the B&R Input and Output blocks.

The problem now is that one input should be used to specify the setpoint for the indoor and outdoor temperature for the last 3 days, and the second input supplies noise values for the respective temperatures. The average temperature should therefore be calculated again for the output for the respective days, taking into account the noise values.

The initial values for setpoint temperature or noise values are specified in the tables below.



|                     | Day 1 | Day 2 | Day 3 |
|---------------------|-------|-------|-------|
| Indoor temperature  | 20    | 22    | 25    |
| Outdoor temperature | 10    | 14    | 9     |

Table 6: Initial values for setpoint temperatures

|                                          | Day 1 | Day 2 | Day 3 |
|------------------------------------------|-------|-------|-------|
| Noise values for the indoor temperature  | 1.4   | 2.1   | 1.8   |
| Noise values for the outdoor temperature | 3.7   | 4.3   | 3.9   |

Table 7: Initial values for noise values

- 1) Use the model from the previous exercise and create the 2-dimensional fields. Enter the initial values from the tables above into the two B&R Input blocks (see [B&R input block](#)). Check the model with the simulation.
- 2) Start the code generation.
- 3) Check the functionality with the external mode.

#### Exercise 8: Use the B&R Workspace block

Use the model from the previous exercise, "Exercise 7: Create 2-dimensional fields" and then use the B&R Workspace block. This was implemented for variables that are already in the MATLAB workspace. The existing variables can easily be selected and used with the help of this block.

For this, you should recreate the temperature input matrix from the previous example in the MATLAB workspace and then select it using the Workspace block. It is important to note the data type for this.

**Note:**

```
>> myMatrix = [20,22,25;10,14,9]
myMatrix =
    20    22    25
    10    14     9
>> myMatrix = single(myMatrix)
myMatrix =
2x3 single matrix
    20    22    25
    10    14     9
```



- 1) Use the model from the previous exercise and use the B&R Workspace block for the temperature input. Check the model with the simulation.
- 2) Start the code generation.
- 3) Determine the difference in the variable declaration in the VAR file of the Automation Studio project.
- 4) Check the functionality with the external mode.

## 9 Creating user libraries

Libraries are used to package software into reusable and compact units. A library is a collection of functions and function blocks, constants and data types. The following section briefly explains the definitions and shows how the function blocks are used. Customized user libraries can then also be created.



Figure 33: Working with libraries

### Library

A library is a collection of functions, function blocks, constants and data types. Any existing library can be inserted into the Logical View at any time. Either B&R standard libraries or libraries created by the user can be selected. In order to be able to reuse program code, it must first be split up into self-contained modules that can be maintained. User libraries are suitable for this purpose. Before the design phase begins, it is important to give some thought to the scope of the individual functional units, i.e. the functions and function blocks. Once this has been done, the interfaces can be declared. Finally, the range of functions can be implemented.

**The following questions must be taken into consideration when creating a library:**

- What is the function of this library?
- Which functions and function blocks are necessary?
- How should the interfaces for the functional units look?
- Will constants and structures be used?
- Are certain things necessary from other libraries to handle certain tasks?
- How will the library be passed on or stored?

### Components of a library

A library consists of several components and properties. A distinction is made between the following components:

- **\*.fun file** - Contains the interface or the structure of the instance of a function or function block.
- **\*.var file** - Contains numeric constants, including the modes that a function block can report, but also numeric parameters that are expected from a function or function block.
- **\*.type file** - Structures that the function block needs internally or structures that must be transferred to the function block in the application.

### Function block

A function block can have multiple return values. The declaration of an instance is necessary. In addition, it is possible for a function block to perform a task while being called multiple times.

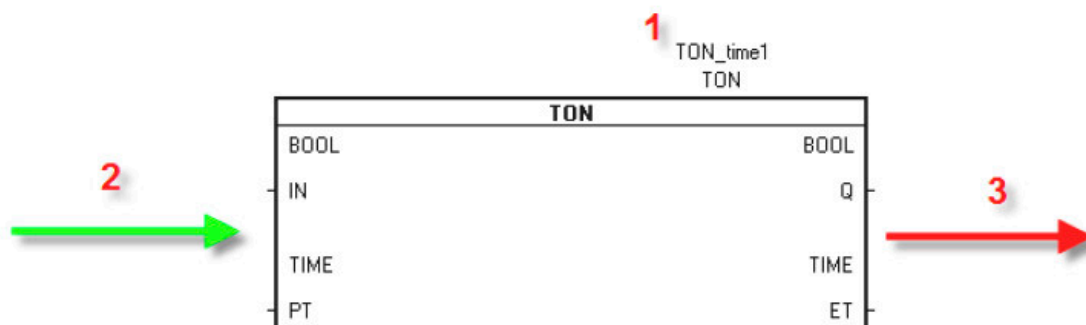


Figure 34: (1) Instance variable, (2) Inputs and (3) Outputs on a function block

- 1) Instance structure, must be declared in the variable declaration editor with a unique name.
- 2) Input parameters are passed directly before or during the function block call.
- 3) Output parameters are written while the function block is being called and can then be used in the program code.

Different instances make it possible to make calculations in tasks using different parameters. An instance can actually be thought of as a structure. The function block takes in the input parameters at the moment it is called and then passes the output parameters on to the instance after the program routine is processed.

#### Code generation for a library with a function block

To generate a library with a function block from a Simulink model, select **Function block** as target in the B&R config block in the **Automation Studio settings** tab. Then it is necessary to specify the name of the library (**Library name**) and of the function block (**Function block name**).

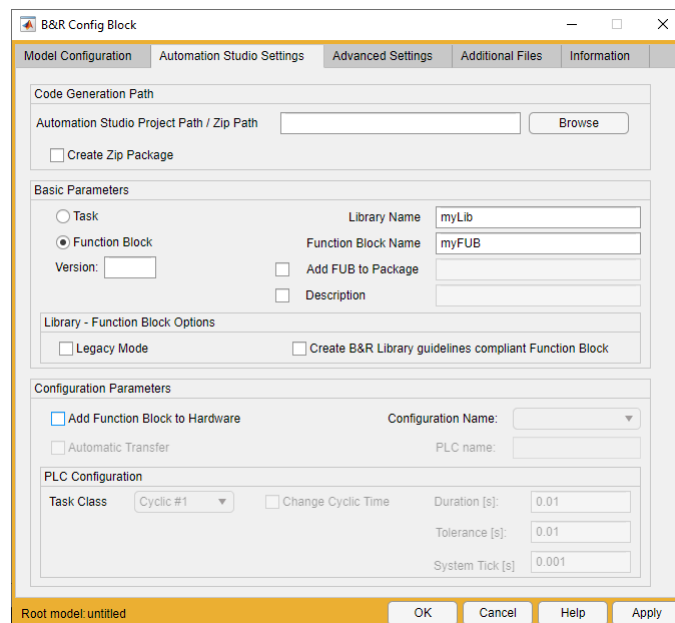


Figure 35: User interface B&R config block - Automation Studio settings

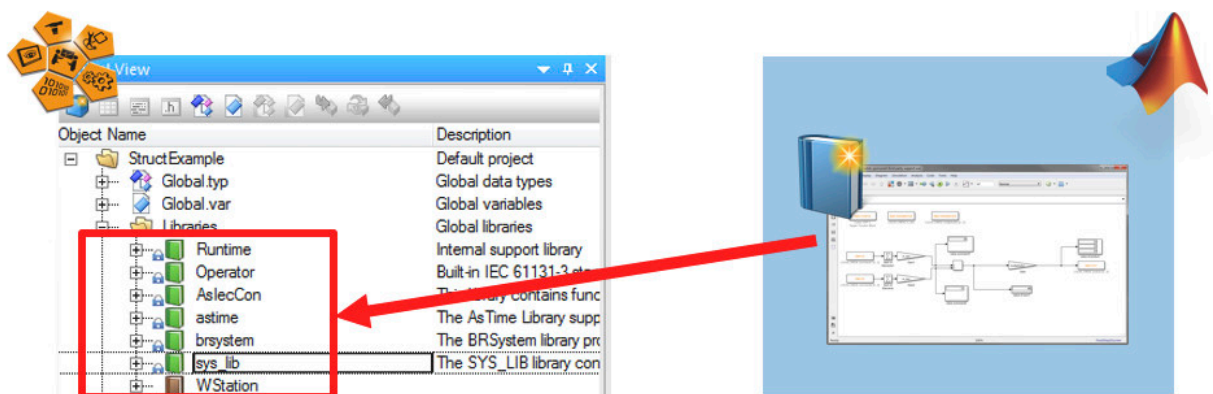


Figure 36: Creating a user library with a function block



The library name is not permitted to be the same as the function block name or the model name. Likewise, the function block name is not permitted to be the same as the library name or model name.

By default, an **ssMethodType** integer variable is created for the inputs and outputs of each function block. This variable determines the status of the function block. The following values can be assigned to the variable:

| Name                                            | Type       | Constant | Replicable | Value | Description [1]               |
|-------------------------------------------------|------------|----------|------------|-------|-------------------------------|
| Constants are now declared as defines in main.c |            |          |            |       |                               |
| SS_INITIALIZE:                                  | SINT := 2; |          |            |       | initialize function block     |
| SS_OUTPUT:                                      | SINT := 3; |          |            |       | run function block cyclically |
| SS_TERMINATE:                                   | SINT := 4; |          |            |       | terminate function block      |

This means that each standard function block must be initialized with `ssMethodType = SS_INITIALIZE` and started with `ssMethodType = SS_OUTPUT`. In addition, the block can be paused/terminated with `ssMethodType = SS_TERMINATE`, i.e. if the block is previously in the state `SS_OUTPUT` and is then changed to `SS_TERMINATE`, the block can be resumed again with `SS_OUTPUT`. The block is reset with `SS_INITIALIZE`.

The **Legacy mode** option causes the function block to be instantiated only once in an Automation Studio project.

The option **Create B&R library guidelines compliant function block** prevents any `ssMethodType` variables from being created; instead the block runs immediately after starting the program. A `bur_init` boolean variable is created instead of `ssMethodType`. The block can be reset by this variable; this variable is edge controlled.

### Exercise 9: Create a user library

Open the demo example 1 from the `bur_demo` examples. Note: Enter `bur_demo (1)` in the command line and change the generation from "Task" to "Function block" in the B&R Config block. In addition, replace a B&R Input block with a Simulink Counter



block:



- 1) Open the demo example 1 from the `bur_demo` examples.
- 2) Change the B&R Config block so that a function block is created.
- 3) Replace a B&R Input block with a Simulink Counter block (e.g. Counter Free-Running).
- 4) Start the code generation.
- 5) Verify the functionality of your function block in Automation Studio by calling the function block in a program.
- 6) Become familiar with the states of the function block (**`ss_Methodtype`**).

# 10 Structures

A structure is composed of individual elements including basic data types, arrays and other structures. The entire structure is addressed via a common name. Each of the individual elements also has its own name. Structures are also known as user data types. Structures are primarily used to group together data and values that have a relationship to each other.

## 10.1 B&R bus block

The **B&R bus input/output blocks** makes it possible to create a process variable whose data type is a structure type. These structure types are already created in Simulink as bus elements via the **bus editor**. This allows structure types to be defined in Simulink, created and transferred to the Automation Studio project.

Figure 37: B&R bus input block

Figure 38: B&R bus output block

**Variable name:** Name of the process variable in Automation Studio.

**Variable description:** Comment entry for the description of process variables.

**Variable scope:** Availability and visibility of the process variable within the entire project. Values: "Global" or "Local".

**Variable data type:** Structure type of the process variable. All available bus elements from Simulink are automatically offered in a drop-down box.

**Memory:** Decides whether the process variable is battery-backed. Values: "Retain" for battery-backed or "Standard" for not battery-backed.

**Array size:** If the value is 1, a process variable is created as a variable. If the value is greater than 1, a process variable is created as a one-dimensional array.



### Bus editor

The bus editor in Simulink can be opened via the command window with command **buseditor**. Various bus elements can then be defined and created here.

Starting with V6.2.1, bus elements can also be defined as a multidimensional array.

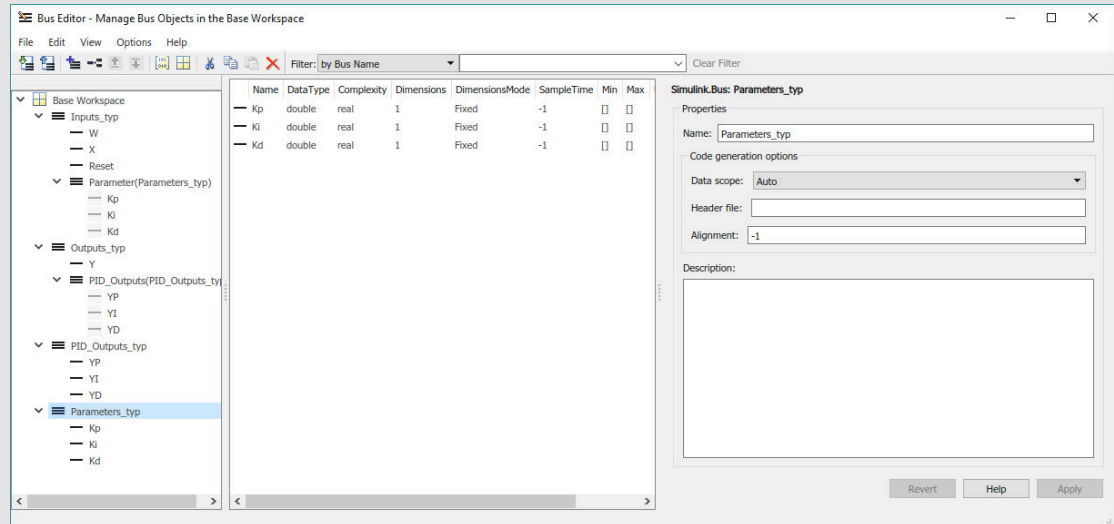


Figure 39: Bus editor in MATLAB

## 10.2 Example: Bus block

### Exercise 10: Create structure types

Use the example `pid_controller_with_bus_object`. You can open the example by executing `bur_demo` in the command window and specifying the appropriate number as a parameter. You may want to save the newly opened example with a shorter filename to avoid path length problems.

- 1) Which bus elements are used in the model?
- 2) Adjust the B&R Config block.
- 3) Generate the Simulink model in Automation Studio.
- 4) Check the functionality by including a trace of the output variable.
- 5) Which variables are responsible for the setpoint specification, return values and reset?

## 10.3 B&R struct block

The **B&R struct input/output block** allows structure types that have already been created in Automation Studio in a .typ file to be used in the Simulink model. The .typ file is read in via block "B&R CONFIG" and made available to the B&R struct block.

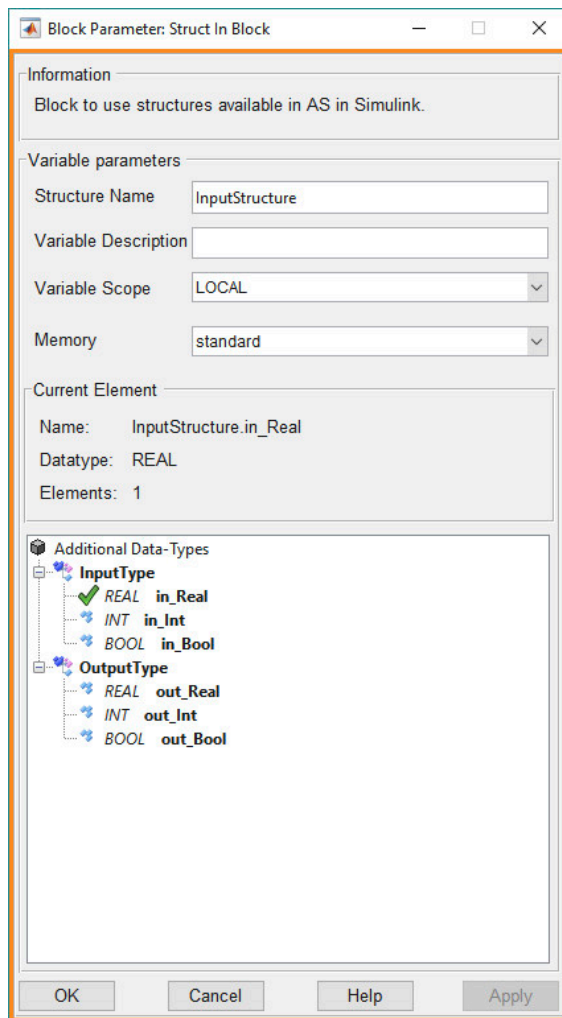


Figure 40: B&amp;R struct input block

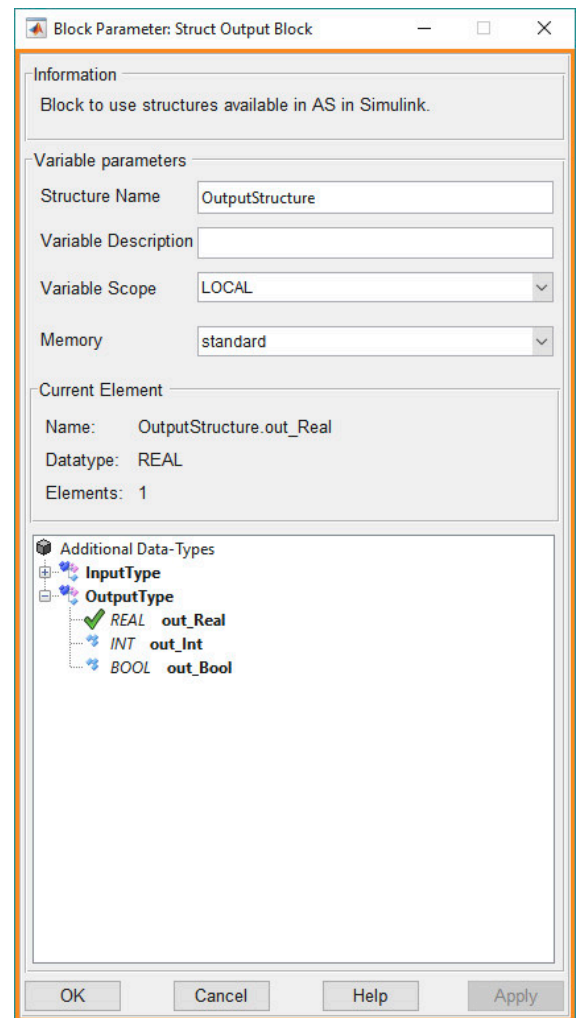


Figure 41: B&amp;R struct output block

**Structure name:** Name of the process variable in Automation Studio.

**Variable description:** Comment entry for the description of process variables.

**Variable scope:** Availability and visibility of the process variable within the entire project. Values: "Global" or "Local".

**Memory:** Decides whether the process variable is battery-backed. Values: "Retain" for battery-backed or "Standard" for not battery-backed.



## 10.4 Example: Structure block

### Exercise 11: Import the structures

Create a model named **structure\_sample**.

#### Declaration

```
TYPE
InputType:STRUCT
    in_Real: REAL;
    in_Int:INT;
    in_Bool:BOOL;
END_STRUCT;
OutputType:STRUCT
    out_Real: REAL;
    out_Int:INT;
    out_Bool:BOOL;
END_STRUCT;
END_TYPE
```

Table 8: Structure declaration in the TYP file of the Automation Studio project

- 1) Create two structures named "InputType" and "OutputType" in an Automation Studio project in the file "Global.typ".
- 2) Link the "Global.typ" file with the Simulink model using the B&R Config block.
- 3) In the model, the respective data types of identical structure elements should be written to each other.
- 4) Start the code generation.
- 5) Check the functionality.

## 10.5 B&R extended struct block

The B&R extended struct input/output block allows entire structures that have already been created in Automation Studio in a .typ file to be used in the Simulink model. The .typ file is read in via block "B&R CONFIG" and made available to the B&R extended struct block. The block generates a Simulink.Bus from an Automation Studio structure and a corresponding MATLAB struct that can contain initial values (the initial values come from the \*.typ file). These variables are created in the base workspace or in a data dictionary. During code generation, a process variable of the type of the selected structure is created in Automation Studio. The corresponding \*.typ file must exist in the Automation Studio project.

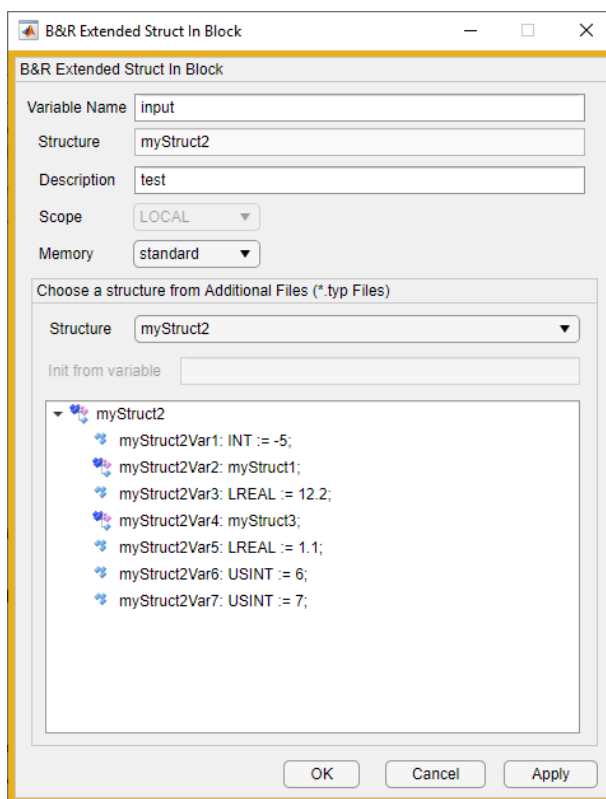


Figure 42: B&amp;R extended struct in block

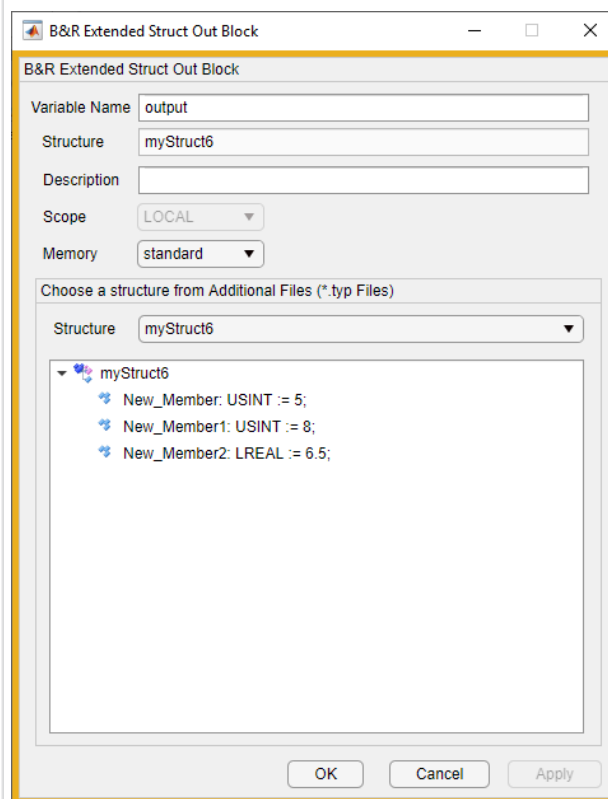


Figure 43: B&amp;R extended struct out block

Table 9: Extended struct block

**Variable name:** Name of the process variable in Automation Studio.

**Structure:** Name of the structure/type of the process variable.

**Description:** Comment entry for the description of process variables.

**Scope:** Availability and visibility of the process variable within the entire project. Values: 'GLOBAL' or 'LOCAL'.

**Memory:** Decides whether the process variable is battery-backed. Values: "Retain" for battery-backed or "Standard" for not battery-backed.



#### Limitations

- This block is only supported by MATLAB R2018a and later
- Only supports basic data types SINT, USINT, INT, UINT, DINT, UDINT, REAL, LREAL, BOOL and one-dimensional arrays
- Initialization from data dictionary not directly supported



In order to initialize from the data dictionary, a MATLAB function block can be connected downstream to the B&R extended struct in block. For more information, please contact B&R Support.

- Currently not possible to mark (memory) the structure as a reference (pointer)
- Initialization currently only possible via .typ file (applies to B&R extended struct in block)

## 10.6 Example: Extended Structure block

### Exercise 12: Read structures with the Extended Structure block

Use the model from the previous exercise (exercise 11): **Read structures**.

- 1) Replace the B&R Structure blocks with B&R Extended Structure blocks.
- 2) What happens in the MATLAB workspace after applying the settings?

- 3) A Bus Selector block or Bus Creator block from the Simulink library is required for a direct connection between a B&R Extended Struct In block and a B&R Extended Struct Out block. (analog to B&R Bus blocks)
- 4) Start the code generation.
- 5) Check the functionality in Automation Studio again.

# 11 Automation Studio hot plugging interface

The **Automation Studio Hot-Plug Interface** block has been available since V6.1.0. Using this block, Simulink models can be connected to existing B&R controllers without having to generate code. This means, for example, that a code-generated controller on a B&R PLC can be connected to a simulated controlled system in MATLAB and then tested. It is possible to use multiple instances in a Simulink model.

The connection is made using the protocol PVI. This protocol is not real-time capable.

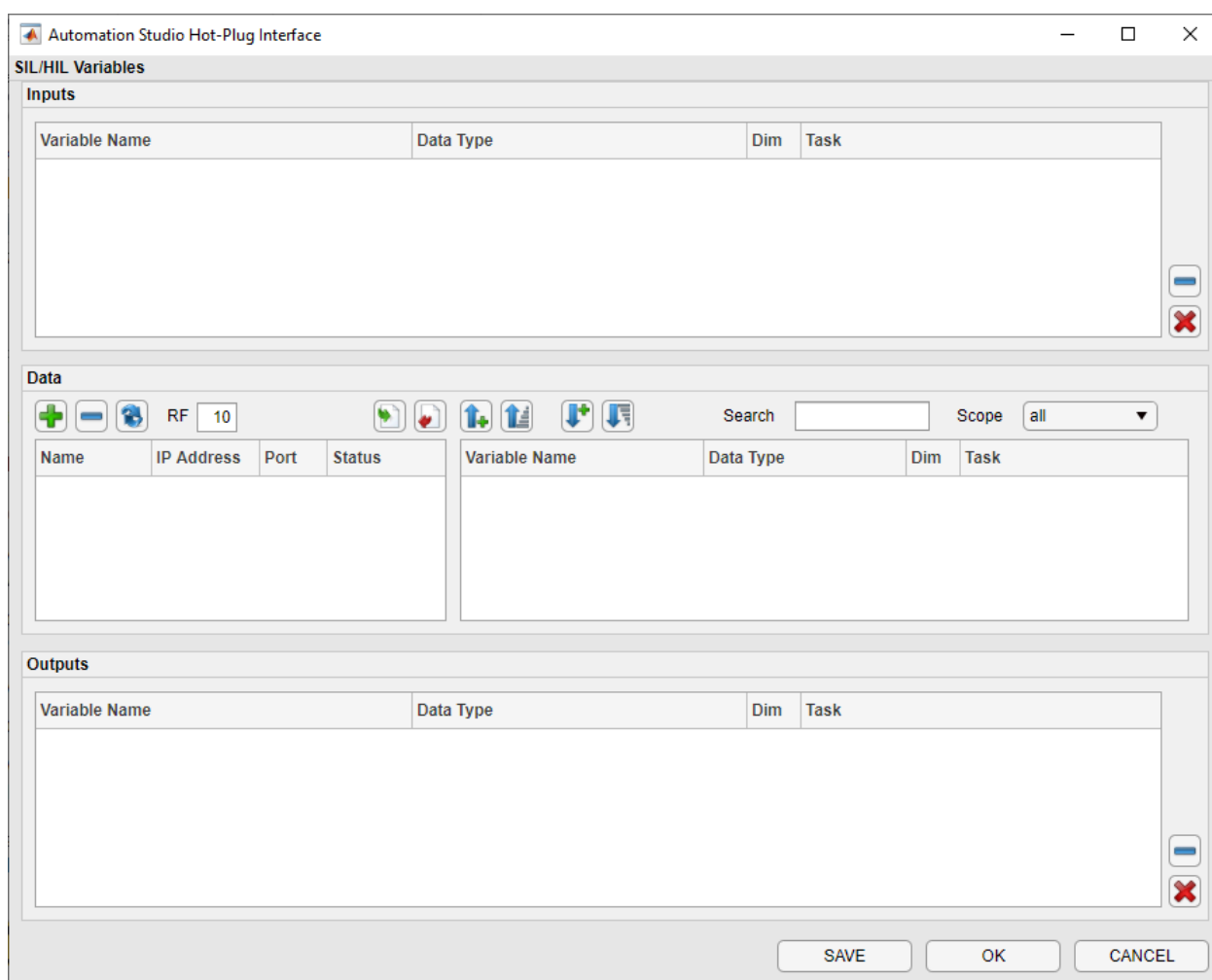


Figure 44: Automation Studio hot plugging interface

## Procedure

- 1) Button **[+]** is used to add a new controller to the left area. If no default settings have been changed, only the correct IP address must be entered.
- 2) The block establishes a connection with the controller via PVI and searches all programs for process variables that are then displayed in the center window on the right.
- 3) Use the up and down arrows to set individual variables as inputs or outputs.
- 4) Click the **OK** button to generate and label the outputs and inputs on the block.
- 5) When simulation is started in Simulink, the block cyclically transfers the inputs to the B&R controllers and receives the outputs again each time a change is made.

Variable names can be searched for directly in the preselection using field **Search**. The programs can be filtered using the **Scope** drop-down box.

A number between -1 and 10000 that stands for the update rate in milliseconds can be entered via field **RF**. This rate indicates how often the variables are updated.



Starting with Automation Runtime version AR 4.61, it has been possible to enable ANSL authentication in an Automation Studio project. Here, a password and role is assigned to authenticated users, which is then assigned to the ANSL authentication under the online parameters in the CPU hardware configuration. For details, see Automation Help. To ensure that the hot plugging interface can now also access the password-protected connection, a dialog box appears every time a new connection is made. The variable list is only reloaded after the username and password have been entered correctly. If this safety option is omitted in the project, i.e. no user is created, the fields in the dialog box are left empty.

In order to be able to test the system appropriately, the time setting in Simulink should be changed to universal real-time.

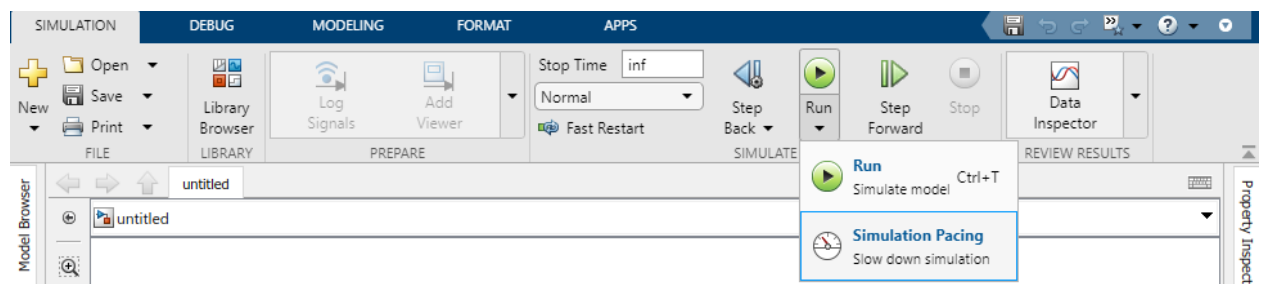


Figure 45: Simulation - Pacing options

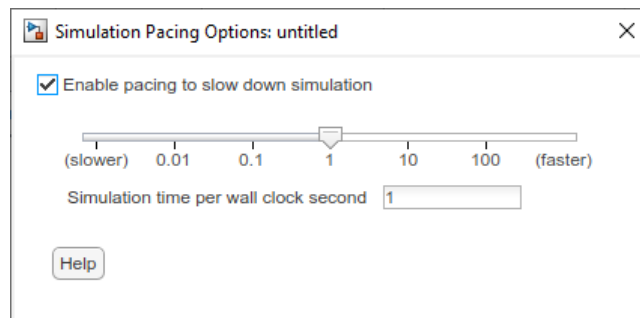


Figure 46: Pacing options - Universal real-time

API hot plugging interface

The hot plugging interface block can be controlled via an API. This makes it possible to automatically communicate with multiple controllers or APCs, for example. This is advantageous if multiple Simulink models with different target hardware should exchange data efficiently via PVI. This is how a central computer (e.g. an APC) with Simulink models and the API of the hot plugging block functions – on the one hand as a distribution node for several end devices, and on the other, as a connection to a cloud-based network (see the following figure).

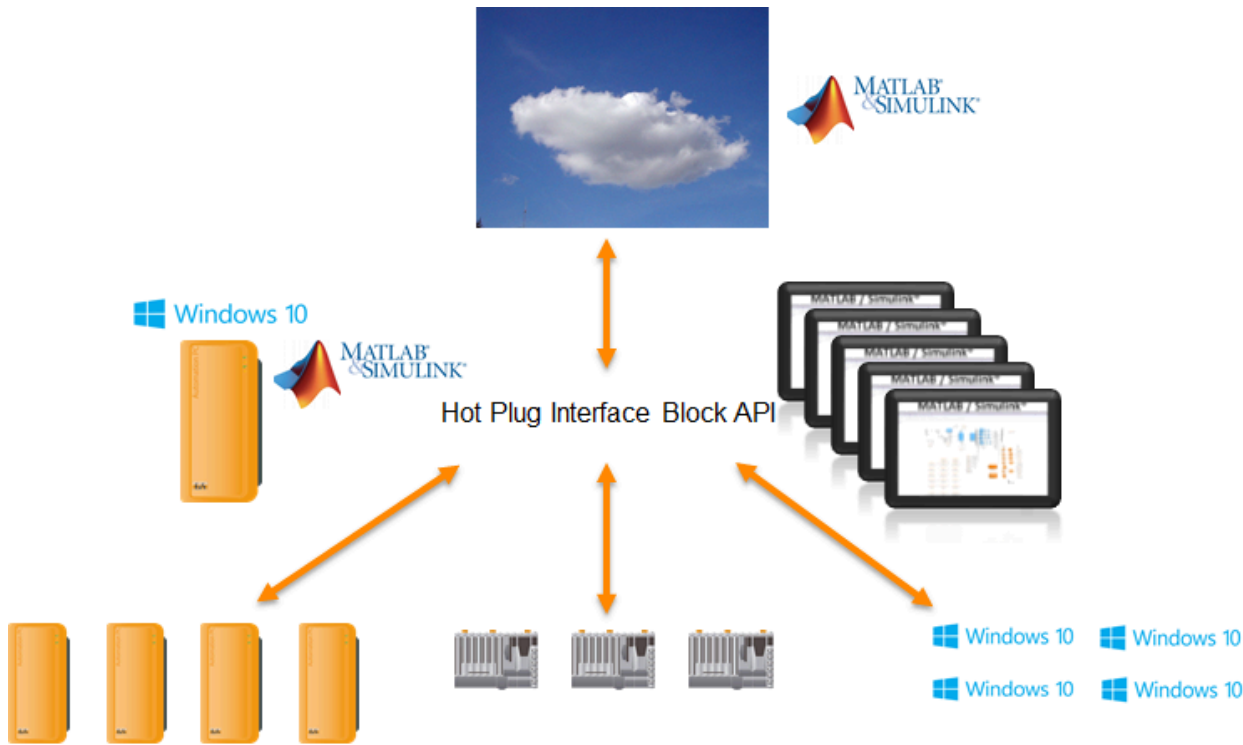


Figure 47: Example of a network application for the API hot plugging interface

In order to obtain an instance of the hot plugging block,

```
myHotPlugBlockInstance = getHotPlugInst('blockName');
```

**getHotPlugInst('blockName')** creates an instance in the MATLAB workspace that can be used to access all functions required for controlling the block automatically via script. The name of the Hot-Plug block for which the API should be enabled must be specified as a parameter. The name of the block can be read from the block's tag (in parentheses). The name can also be user-defined (direct input of the name above the tag).



All functions at a glance:

| Function name                                                                              | Description                                    |
|--------------------------------------------------------------------------------------------|------------------------------------------------|
| addTarget('targetname', 'ipadress', 'port', 'user-name' (optional), 'password' (optional)) | Adds a connection to a target hardware.        |
| removeTarget('targetname')                                                                 | Removes a connection.                          |
| getTargetList()                                                                            | Returns a list of all created connections.     |
| refreshTarget()                                                                            | Restarts the connection (e.g. after a timeout) |
| getVarList()                                                                               | Returns a list of all PVI variables.           |

Table 10: API functions of the hot plugging block

| Function name                                                        | Description                                                                                     |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>getVars()</code>                                               | Returns a list of all variables that are not assigned to an Inport or Outport block.            |
| <code>addToInPort('variablename1', 'variablename2',...)</code>       | Assigns variables to the Inport block.                                                          |
| <code>getInPortVars()</code>                                         | Returns a list of all variables assigned to the Inport block.                                   |
| <code>addToOutPort('variablename1', 'variablename2',...)</code>      | Assigns variables to the Outport block.                                                         |
| <code>getOutPortVars()</code>                                        | Returns a list of all variables assigned to the Outport block.                                  |
| <code>addAllToInPort()</code>                                        | Assigns all variables to the Inport block that are currently not assigned to the Outport block. |
| <code>addAllToOutPort()</code>                                       | Assigns all variables to the Outport block that are currently not assigned to the Inport block. |
| <code>removeFromInPort('variablename1', 'variablename2',...)</code>  | Removes variables from the Inport block.                                                        |
| <code>removeFromOutPort('variablename1', 'variablename2',...)</code> | Removes variables from the Outport block.                                                       |
| <code>removeAllFromInPort()</code>                                   | Removes all variables from the Inport block.                                                    |
| <code>removeAllFromOutPort()</code>                                  | Removes all variables from the Outport block.                                                   |
| <code>connectToTarget('targetname')</code>                           | Connects to target hardware 'targetname'                                                        |
| <code>setRF(value)</code>                                            | Sets the refresh time for updating the variables in milliseconds (default 10 ms).               |
| <code>getRF()</code>                                                 | Outputs the refresh time in milliseconds.                                                       |

Table 10: API functions of the hot plugging block

**Example:**

```

myHotPlugBlockInstance = getHotPlugInst('bur_hp');
myHotPlugBlockInstance.addTarget('myHost', '127.0.0.1', '11169');
myVars = myHotPlugBlockInstance.getVars();
myHotPlugBlockInstance.addToInPort(myVars(1).name, myVars(2).name);
myHotPlugBlockInstance.addToOutPort(myVars(3).name, myVars(4).name);

```

## 12 Example: Automation Studio Hot-Plug Interface block

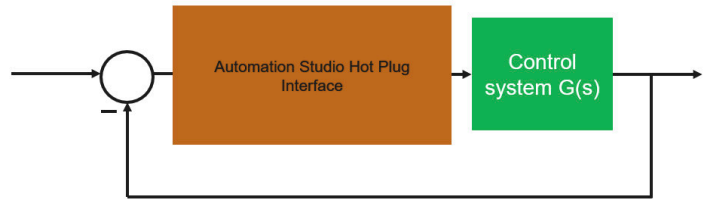
### Exercise 13: Create a control loop for a temperature-controlled system

This exercise is intended to show a practical application of the Hot-Plug Interface block. The adjacent graphic shows a closed control loop. The Hot-Plug Interface block should be used for a co-simulation between the controller (Automation Studio project) and the DUT (Device Under Test, MATLAB/Simulink).

The green box marked **Control system** is intended to represent a controlled system, i.e. the DUT (Device Under Test).

The orange box should represent the controller. A PID controller from the demo examples should be used for this purpose (Note: Enter `bur_demo(2)` in the command line). This controller should be generated in an Automation Studio project.

Here, the controlled system should be a temperature model from the demo examples (Note: Enter `bur_demo(3)` in the command line). This is **not** code generated.



- 1) Open the PID controller model with `bur_demo(2)`.
- 2) Start the code generation of this model.
- 3) Close the model in Simulink.
- 4) Open the temperature model with `bur_demo(3)`.
- 5) Add the Hot-Plug Interface block.
- 6) Configure the Hot-Plug Interface block according to the block diagram above. The output of the block is the input of the temperature model; the input of the block is the output of the temperature model. The output of the temperature model is the feedback variable to the input of the PID controller.



# 13 Model structuring and data management in MATLAB

This section deals with the targeted use of the options available for preparing, managing and structuring Simulink models and their data.

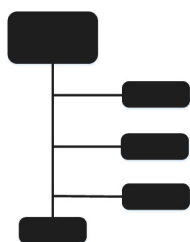


Figure 48: Subsystems

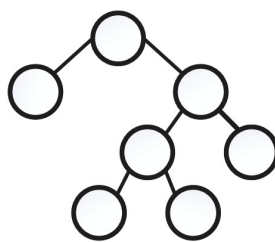


Figure 49: Model referencing



Figure 50: Data dictionary

Correct usage allows software errors to be avoided and the flexibility and consistency of the application to be improved. The information provided here provides a small insight into which tools and procedures can be used. In addition, an overview of possible storage formats and locations for data should be a decision-making aid for tasks to be solved.

## 13.1 Subsystems

You can use **subsystems** to create a hierarchical model with many levels. A subsystem is a group of blocks that you replace with a single subsystem block. This system structures a complex model and holds together the functionality of the associated blocks.

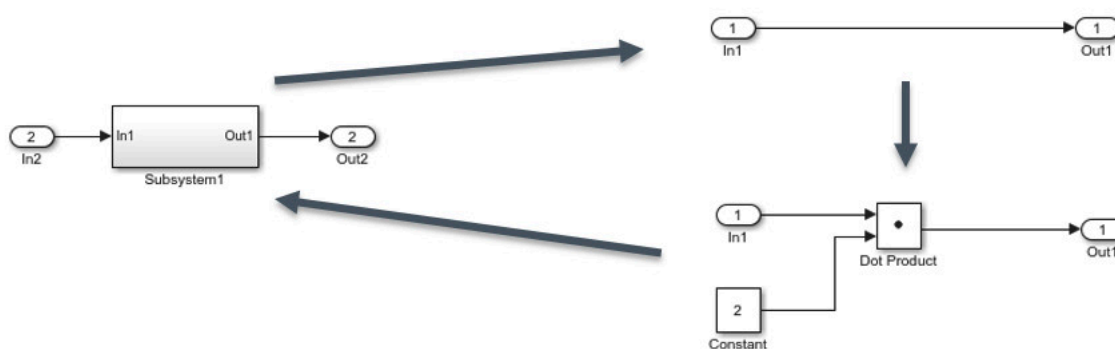


Figure 51: Replacing a group of blocks with a subsystem and encapsulation of the functionality.

There are different types of subsystems. For more information about each subsystem, see the MATLAB documentation.

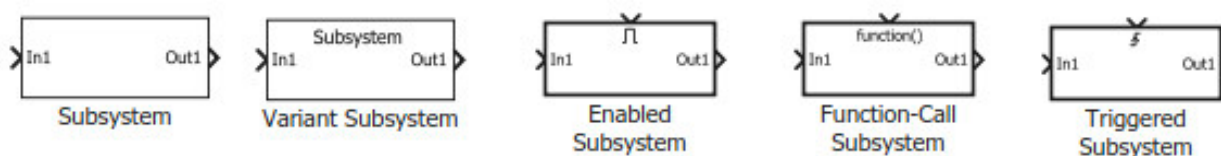


Figure 52: Selection of subsystem blocks.

The following approaches are available to create a subsystem:

- Adding a subsystem block to the model. Blocks can be added to this after it is opened.
- Marking all blocks that should form a subsystem, right-click and select "Create subsystem from selection".
- Copying a model directly into a Simulink subsystem window
- Copying an existing subsystem into a model

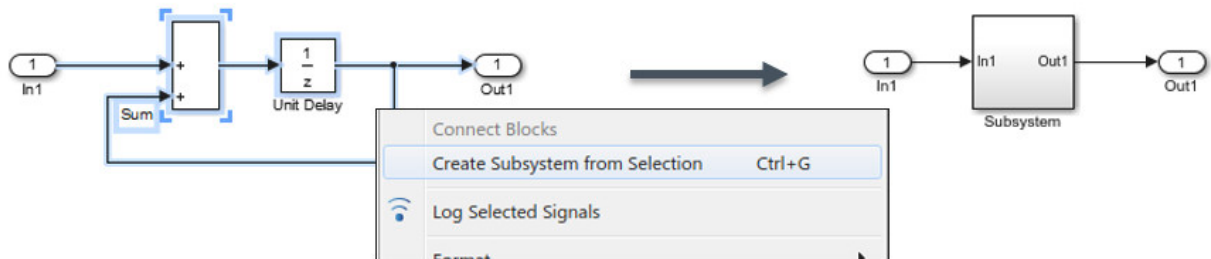


Figure 53: Creating a subsystem using function **Create subsystem from selection**.



For additional information about subsystems, see the MathWorks website.  
[www.mathworks.com/help/simulink/ug/creating-subsystems.html](http://www.mathworks.com/help/simulink/ug/creating-subsystems.html)  
[www.mathworks.com/help/simulink/ug/configure-a-subsystem.html](http://www.mathworks.com/help/simulink/ug/configure-a-subsystem.html)  
[www.mathworks.com/help/simulink/examples/variant-subsystems.html](http://www.mathworks.com/help/simulink/examples/variant-subsystems.html)

## 13.2 Example: Subsystems

### Exercise 14: Subsystems

Open the example **Inverted\_Pendulum\_with\_variant\_Subsystems** using `bur_demo(11)` and answer the following questions:

- What type of subsystem was used?
- How is it possible to navigate between the different levels?
- Generate a hierarchical view of the model.



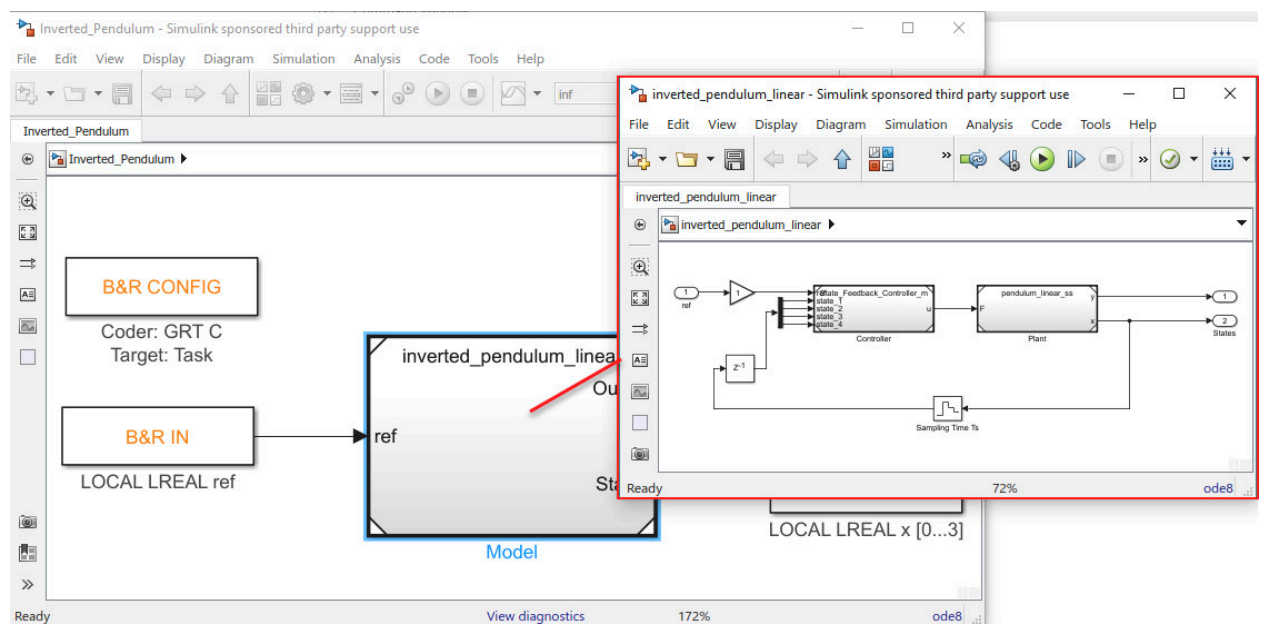
Simulink's **model dependency viewer** maps the hierarchical structure of a Simulink model.

## 13.3 Model referencing

You can use **model referencing** to integrate models into another model using a model block. Simulink offers the two model blocks "Model" and "Model variants" for referencing models. The difference between the two blocks is that the user can use "Model variants" to set different references to other models. The user can then switch between the different referenced models. Block "Model" can only set one model reference.

Model referencing allows you to do the following, for example:

- Structure large models hierarchically
- Reuse the same functionality without having to redefine it
- Modular development
- Incremental loading of models
- Incremental code generation of the model



For additional information about model referencing, see the MathWorks website.  
[www.mathworks.com/help/simulink/model-reference.html](http://www.mathworks.com/help/simulink/model-reference.html)  
[www.mathworks.com/help/simulink/ug/overview-of-model-referencing-1.html](http://www.mathworks.com/help/simulink/ug/overview-of-model-referencing-1.html)  
[www.mathworks.com/help/simulink/ug/model-referencing-limitations.html](http://www.mathworks.com/help/simulink/ug/model-referencing-limitations.html)



B&R input and output blocks within a referenced model are not created as process variables!

## 13.4 Model referencing example

### Exercise 15: Model referencing

Open the example **Inverted\_Pendulum** using `bur_demo(10)` and answer the following questions:

- What is the difference to the subsystem?
- How many sub-level models does the top-level model have?
- Generate a hierarchical view of the model.



Simulink's **model dependency viewer** maps the hierarchical structure of a Simulink model.

## 13.5 Data dictionary

A **data dictionary** is a persistent repository of data that is relevant for a model. Persistent means that no data must be reloaded during development. A data dictionary offers more functions than the workspace. A complete list is available [here](https://www.mathworks.com/help/simulink/ug/what-is-a-data-dictionary.html):

<https://www.mathworks.com/help/simulink/ug/what-is-a-data-dictionary.html>.

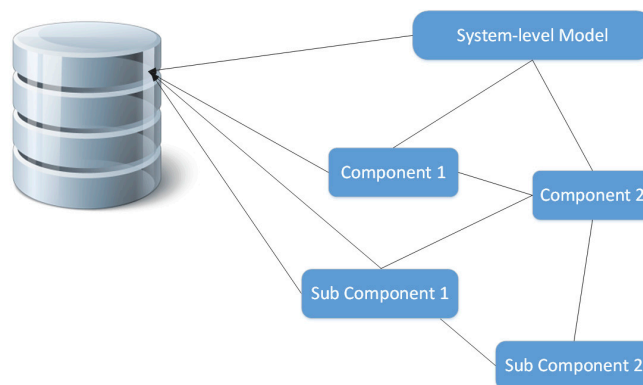


Figure 55: Possible link between the Simulink model and data dictionary.

The main difference to the workspace is the persistence of data, i.e. the data is always available and does not have to be reloaded during development. The data dictionary stores design data (e.g. parameters) and not simulation data that is input or output from a model simulation that enters and exits input and output blocks. In the **Model Explorer**, tab "**Data**" allows the model to link to either the workspace or a data dictionary:

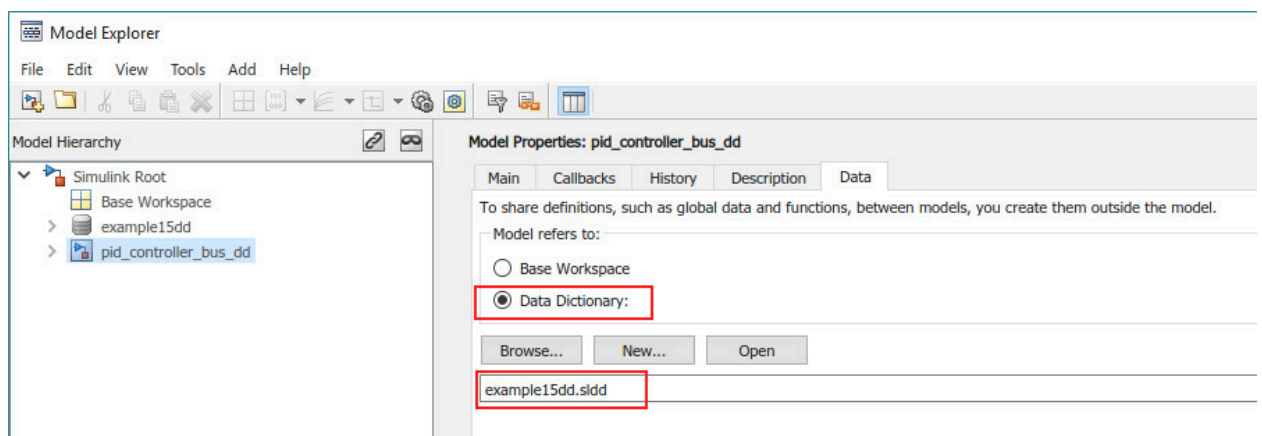


Figure 56: Model Explorer - Setting a link to the data dictionary.

After the setting has been applied, the data dictionary icon appears in the model hierarchy. If you open section "Design data", all variables that were in the workspace are displayed. New variables can be added there and existing variables edited and deleted.

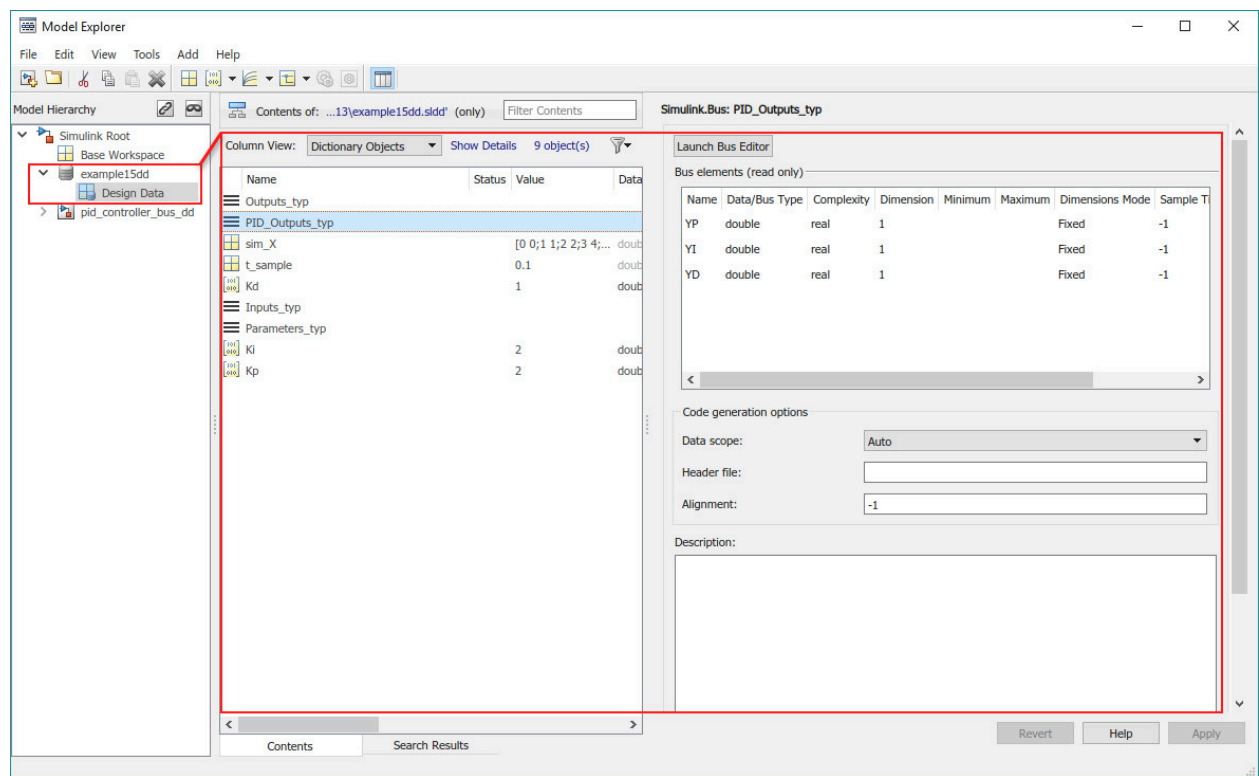


Figure 57: Model Explorer - Adding variables to the data dictionary.

MATLAB supports data dictionaries in R2014a and higher. Automation Studio Target for Simulink supports data dictionaries in V5.6.0 or later with the B&R parameter block, B&R workspace variable block and B&R bus input/output block.

By linking a model to a Data Dictionary, the following automatically occurs:

- Variables of the B&R parameter block are written to the data dictionary.
- In the B&R workspace variable block, a list of variables is only offered from the data dictionary.
- In the B&R bus input/output block, a list of bus elements is only offered from the data dictionary.



As long as a model is connected to a data dictionary, the B&R blocks only support the variables created in the data dictionary.

## 13.6 Data dictionary example

### Exercise 16: Data dictionary

Open the example **pid\_controller\_bus\_dd** using `bur_demo(15)`.

- Link the model to the attached data dictionary.
- Adjust the B&R config block.
- Generate the Simulink model in Automation Studio.
- Check the functionality by including a trace of the output variable. Does this reach the final value?

# 14 Summary

Simulation and model-based development are essential aspects when working with Automation Studio Target for Simulink. The interface allows easy conversion of existing Simulink models and guarantees seamless integration of the generated program code into an existing Automation Studio project.

Automation Studio Target for Simulink thus extends the simulation capabilities of Automation Studio by integrating the MATLAB/Simulink simulation tool.

# 15 Appendix

## 15.1 Programming interfaces (API)

### B&R Config block

There is an API for the B&R Config block that is used to control it programmatically, i.e. from the command line or a .m file.

Example for setting the Automation Studio project path:

```
bur_set_param('bur_projpath', 'C:\test');

bur_get_param('bur_projpath')
>> ans =
    'C:\test'
```

To do this, the block must be added to the model and the model window must be active. The command `gcs` can be used to check which model window is active.

For all API commands, see TM140.

### B&R Automation Studio Hot-Plug Interface block

There is an API for the B&R Automation Studio Hot-Plug Interface block that is used to control it programmatically, i.e. from the command line or a .m file.

First, an instance of the block must be created. The block name must be entered here since there can be multiple Hot-plug blocks in a model:

```
myHotPlugBlockInstance = getHotPlugInst('bur_hp');
```

Afterwards, the API functions can be used:

```
myHotPlugBlockInstance = getHotPlugInst('bur_hp');
myHotPlugBlockInstance.addTarget('myHost', '127.0.0.1', '11169');
myVars = myHotPlugBlockInstance.getVars();
myHotPlugBlockInstance.addToInPort(myVars(1).name, myVars(2).name);
myHotPlugBlockInstance.addToOutPort(myVars(3).name, myVars(4).name);
```

For all API commands, see TM140.

## 15.2 Data type conversions

During programming, it may become necessary to convert one data type to another.

When assigning a variable of a data type with a smaller range of values to one with a larger range, implicit conversion is carried out. When the opposite is done (the range of values becomes smaller), the user has to handle the conversion in the program code itself, i.e. explicitly.



The IEC 61131-3 conversion functions are contained in the **AslecCon** library, which is automatically included in a new Automation Studio project.

### Implicit data type conversion

Implicit data type conversion is type conversion from one data type to another that is performed by the compiler itself.

### Explicit data type conversion

If the value a data type with a larger range of values is assigned to a data type with a smaller range, then it's up to the user to carry out the conversion. The functions in the "AslecCon" library can be used to carry out the conversion.

### Direct composition and subranges

In addition to arrays, other derived data types can also be derived from basic data types. It is possible to derive simple data types directly. A new data type with a new name is created that has the same properties as the basic data type. An initial value can also be assigned to the new data type. All variables that use these data types therefore have the



configured value. A value range can also be specified for direct derivation. It is therefore only possible to assign values to variables of this type that lie within the set value range. A subrange can also be assigned to variables.

|                                                                                   |                                  |                                                            |
|-----------------------------------------------------------------------------------|----------------------------------|------------------------------------------------------------|
|  | <b>Variable with a subrange</b>  | <pre>VAR     varSubRange : USINT (24..48); END_VAR</pre>   |
|                                                                                   | <b>Data type with a subrange</b> | <pre>TYPE     Voltage_typ : USINT (12..24); END_TYPE</pre> |

Table 11: Declaring a variable and data type with a subrange

15.2.1 B&R extended I/O block

The **B&R extended input/output block** generates a process variable in Automation Studio that is subject to data type conversion and/or direct derivation. The conversion or casting is carried out automatically by the block. One use case would be to convert hardware inputs or outputs (usually INT) to floating-point values (REAL or LREAL) for more powerful calculations in the controller algorithm and vice versa.

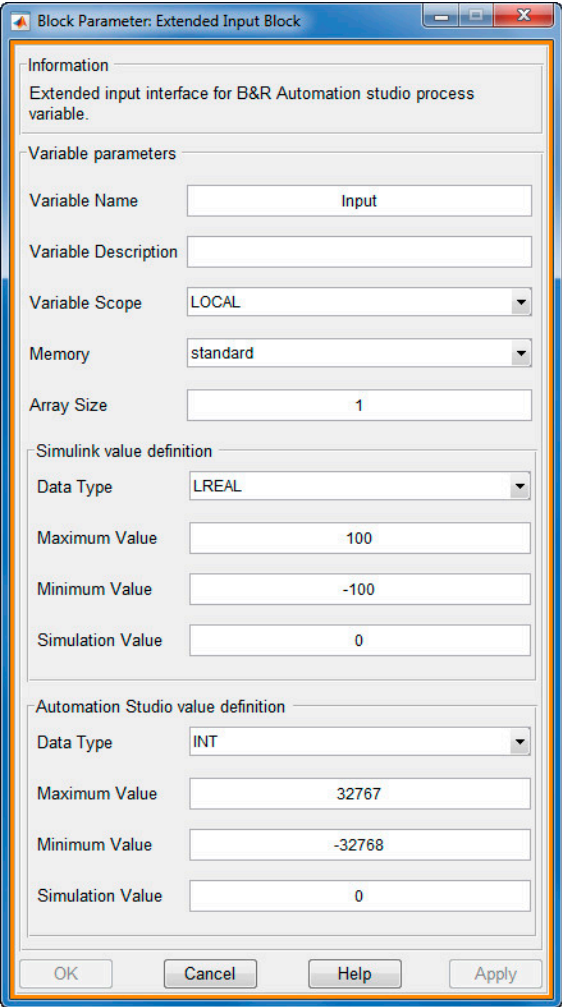


Figure 58: B&R extended input block

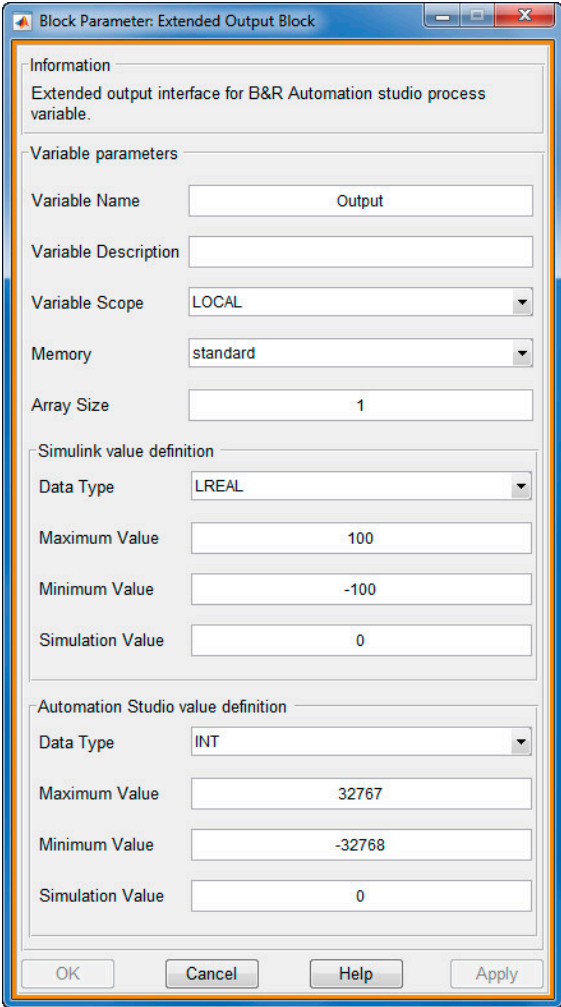


Figure 59: B&R extended output block

- Variable name:** Name of the process variable in Automation Studio.
- Variable description:** Comment entry for the description of process variables.
- Variable scope:** Availability and visibility of the process variable within the entire project. Values: "Global" or "Local".
- Memory:** Decides whether the process variable is battery-backed. Values: "Retain" for battery-backed or "Standard" for not battery-backed.

**Array size:** If the value is 1, a process variable is created as a variable. If the value is greater than 1, a process variable is created as a one-dimensional array.

**Automation Studio data type:** Data type of the process variable per IEC 61131-3 that is available in Automation Studio. With reference to the B&R extended output block, the conversion from the Simulink data type to the Automation Studio data type is calculated as follows:

$$AS\_value = (AS\_DataType) \left[ AS\_Min + \frac{(AS\_Max - AS\_Min)}{(SI\_Max - SI\_Min)} (SI\_value - SI\_Min) \right]$$

**Automation Studio minimum value:** Minimum value that the Automation Studio variable is permitted to accept.

**Automation Studio maximum value:** Maximum value that the Automation Studio variable is permitted to accept.

**Automation Studio simulation value:** Initial value of the process variable, which is limited by the minimum and maximum values.

**Simulink data type:** Data type of the process variable per IEC 61131-3 that is available in Automation Studio.

With reference to the B&R extended input block, the conversion from the Automation Studio data type to the Simulink data type is calculated as follows:

$$SI\_value = (SI\_DataType) \left[ SI\_Min + \frac{(SI\_Max - SI\_Min)}{(AS\_Max - AS\_Min)} (AS\_value - AS\_min) \right]$$

**Simulink minimum value:** Minimum value that the Automation Studio variable is permitted to accept.

**Simulink maximum value:** Maximum value that the Automation Studio variable is permitted to accept.

**Simulink simulation value:** Initial value of the process variable, which is limited by the minimum and maximum values.

## 15.3 Diagnostics

This section explains the different variants of diagnosis the Automation Studio project using the Simulink model. Automation Studio provides the user with a wide variety of diagnostic tools for installation and localizing errors. They are described in more detail in section [Diagnostic tools](#).

### 15.3.1 Debugging

When option "Debugging" is enabled, the generated task/library is marked as "debug-bar" in the Automation Studio project.

Advanced Parameters

☐ Create global var-File

☐ Simulink Web Interface

☒ Debugging

☐ Create Simulink I/O as variable

☐ Create as global variable

☐ Pre Processing File:

☐ Post Processing File:

☐ Additional Compiler Switches:

☐ Password for Code Encryption:

Heap Size:

☐ Allow I/O variable duplicates

☐ Silent Mode

☐ Create Simulink.Parameter as variable

☐ Create as global variable

6553500

Figure 60: Advanced settings - Debugging

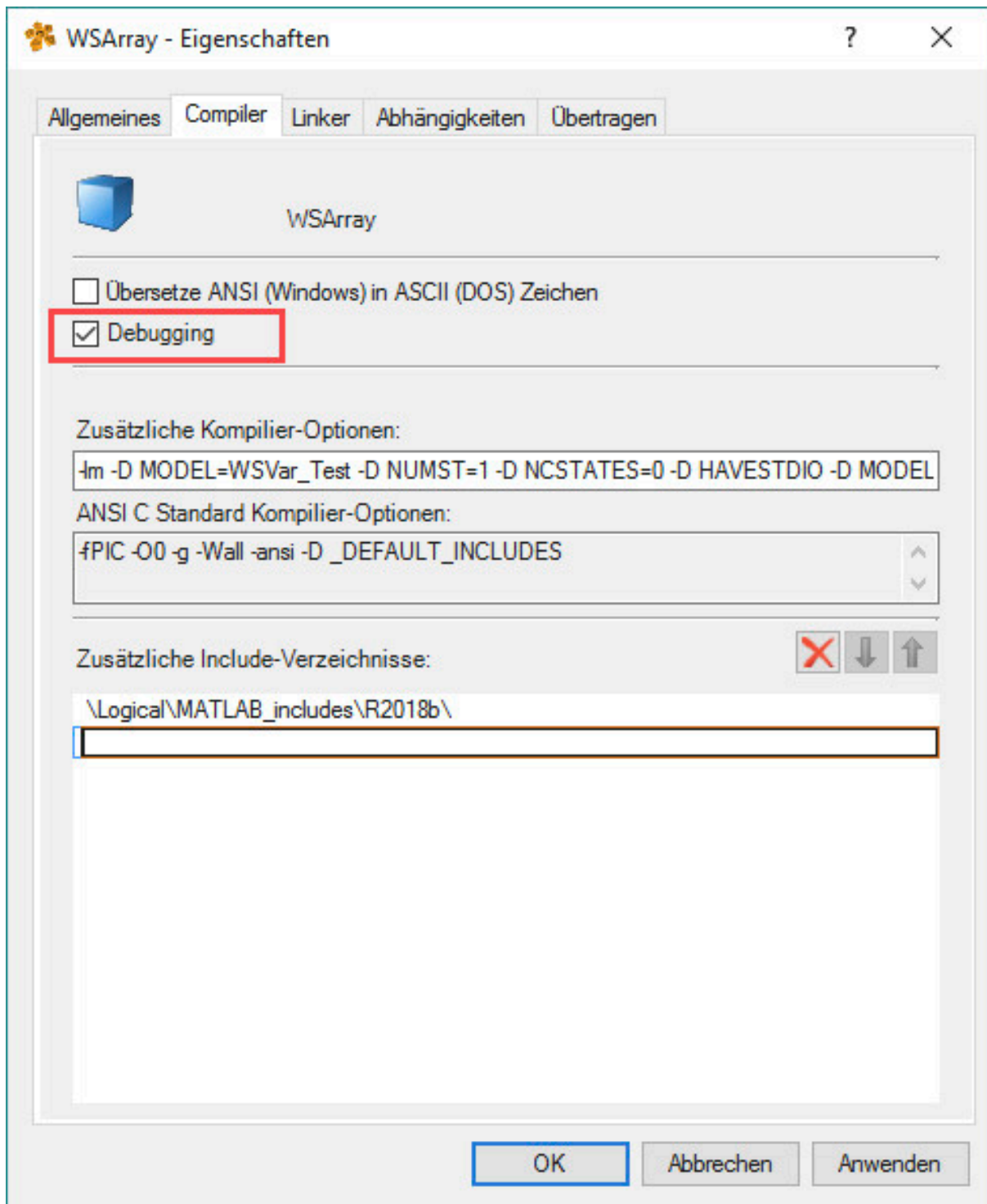


Figure 61: Automation Studio task properties - Tab compiler

### 15.3.2 External mode

The Simulink **external mode** configuration setting enables communication at runtime between the Simulink model and the code-generated model running on the target system.

External mode establishes one communication service each on the system on which MATLAB is installed (PC) and the target system (controller or PC). These two services on the respective systems establish a communication channel between the Simulink engine and the generated code provided on the target system. The communication service isolates the model process on the target system from the code and transport layer. The tasks of the transport layer are to format, transmit and receive data packets. The communication service on the host computer receives the data packets through the transport layer and updates the Simulink model display. The diagram shows the connection that the

communication service establishes in external mode between Simulink on the host computer and the provided code on the target system.

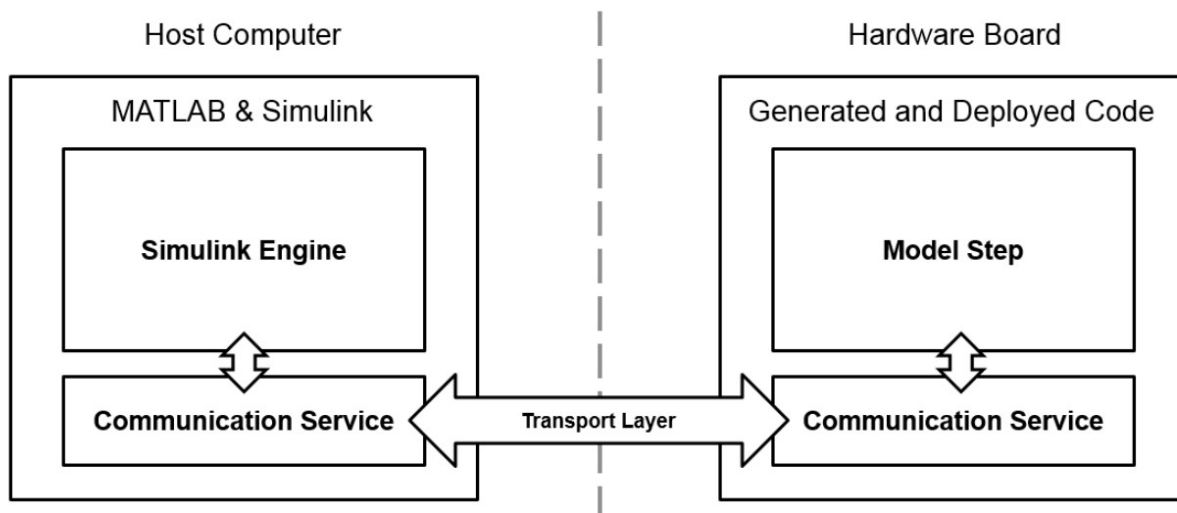


Figure 62: External mode communication scheme MATLAB/Simulink

### Configuring external mode

To use external mode, the feature must be enabled in the B&R config block in tab **Advanced settings**. In addition, three settings must be made:

- **IP address:** IP address of the target system. The "localhost" IP address 127.0.0.1 is entered as the default value (e.g. when using ARsim).
- **Port:** External mode uses the port for communication with Automation Studio. The port number is an integer value between 1024 and 65535. The MATLAB default value is port number 17725.
- **Buffer size:** Buffer size that is preallocated on the target system. Default value: 1000000 bytes (1 MB)

B&R libraries **AsArLog** and **AsTCP** are necessary for external mode in Automation Studio. They are automatically added to the Automation Studio project when a task or function block is created. If a ZIP package is created, these libraries must be added manually in the Automation Studio project.

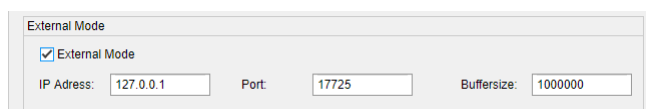


Figure 63: External mode settings in the B&R config block

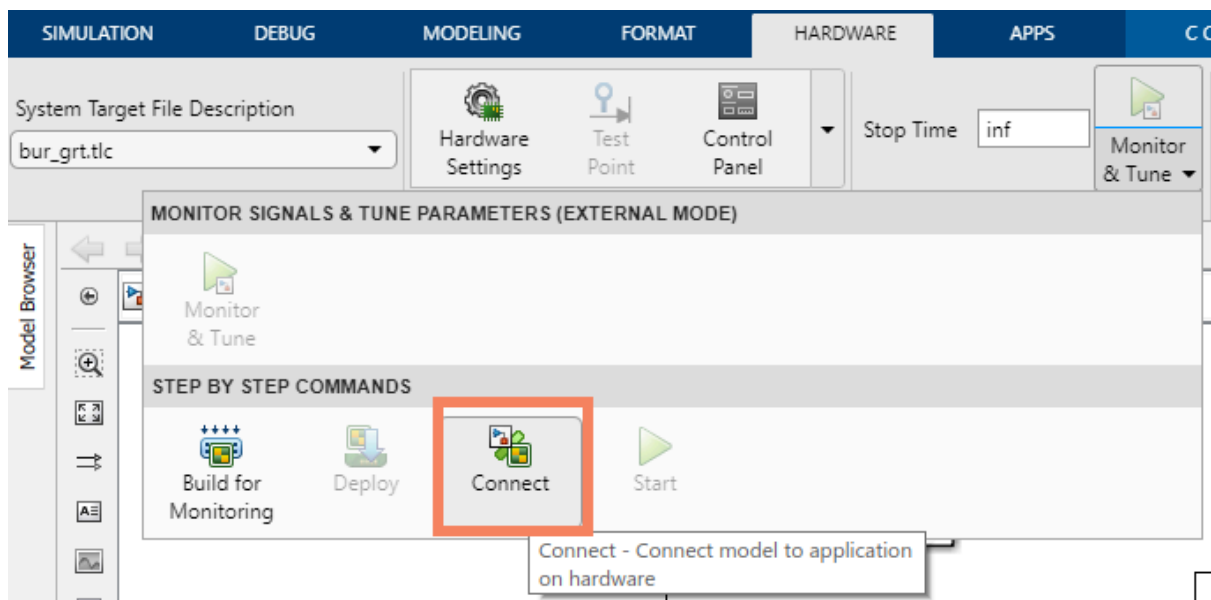


Enabling external mode generates additional code during model code generation that affects performance. For additional information, see the documentation for [External mode](https://de.mathworks.com/help/rtw/ug/set-up-and-use-hosttarget-communication-channel.html) from MATLAB.

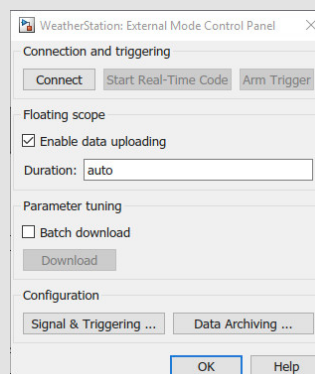
**Link:** <https://de.mathworks.com/help/rtw/ug/set-up-and-use-hosttarget-communication-channel.html>

### Starting external mode

- 1) Enable and specify external mode features in the B&R Config block.
- 2) Generate code and start the target system.
- 3) Configure the Simulink model for external mode and start the external mode connection.



External mode can also be started and stopped via the **External Mode Control Panel** with the connect/disconnect button.



### Manipulating variables on the target system from Simulink

While communication between the Simulink model and target system is taking place, process variables can be observed and manipulated in the Simulink model.

#### Displaying process variables in the Simulink model

To check current values in the Simulink model at runtime, blocks from Simulink library "Sinks" can be used (e.g. "Scope" or "Display").

#### Manipulating process variables in the Simulink model

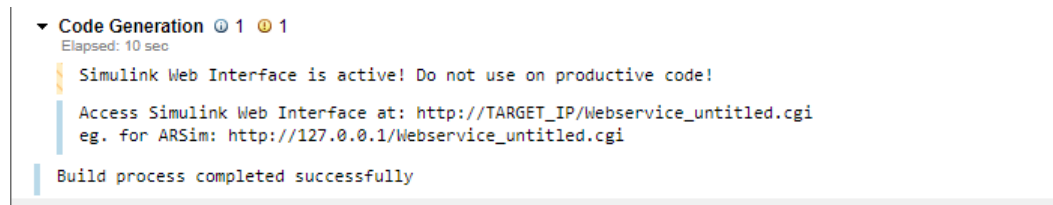
There are two ways to manipulate parameters from Simulink. The first is modifying the values of Simulink blocks like **Gain** or **Constant** directly during the external mode connection. The other is to use the **External Mode Control Panel**. The External Mode Control Panel allows values of workspace variables to be changed locally in Simulink and then downloaded to the target system using command **Batch download**. This can be used with the B&R Parameter block or B&R Workspace Variable block, for example. Only parameters that are located in the MATLAB workspace require the batch download. (B&R input/output parameters cannot be changed because they are not in the workspace, for example) The External Mode Control Panel is located under Code > External Mode Control Panel > Batch download > Download.

## 15.3.A Simulink web interface

It is possible to operate the Simulink model via a web service in the web browser. This feature is only available when a task is generated.

The web service can be accessed at the following address:

`http://<TARGET_IP>/Webservice_<ROOT MODEL NAME>.cgi`



```

▼ Code Generation ⓘ 1 ⓘ 1
Elapsed: 10 sec

Simulink Web Interface is active! Do not use on productive code!

Access Simulink Web Interface at: http://TARGET_IP/Webservice_untitled.cgi
eg. for ARSim: http://127.0.0.1/Webservice_untitled.cgi

Build process completed successfully

```

Figure 64: Simulink web interface - Code generation warning

### The current version supports the following Simulink blocks

- # All B&R blocks (except for the B&R Extended Struct block)
- # Simulink Inport and Outport blocks
- # **Gain, Sum, Product, Saturation** (if not optimized away by the coder)
- # Subsystems
- # Stateflow

### The following data types are supported:

- # Standard data types: LREAL, REAL, SINT, USINT, INT, UINT, DINT, UDINT, BOOL, USINT
- # One-dimensional arrays of standard data types
- # Bus/structures with standard data types as elements



Enabling the Simulink web interface generates additional code during model code generation that affects performance. In addition, all process variables used can be read and written via a non-secure http access. This feature should therefore only be used for testing and debugging a model.

**B&R Config Block**

Model Configuration | Automation Studio Settings | **Advanced Settings** | Additional Files | Information

**External Mode**

☐ External Mode

IP Address:  Port:  Buffersize:

**Advanced Parameters**

☐ Create global var-File ☐ Allow I/O variable duplicates

☒ Simulink Web Interface ☐ Silent Mode

Referenced model depth  ▼

Subsystem depth (from top model)  ▼

☐ Debugging

☐ Create Simulink I/O as variable ☐ Create Simulink.Parameter/Signal as variable

☐ Create as global variable ☐ Create as global variable

☐ Pre Processing File:

☐ Post Processing File:

☐ Additional Compiler Switches:

☐ Password for Code Encryption:

Heap Size:

Root model: depthTest

OK Cancel Help Apply

Figure 65: When activating the Simulink web interface, it is also possible to specify the hierarchical depth up to which referenced models in the system or subsystems in the referenced models (and also the Top model) should be displayed in the web interface. For the setting of the depth of the subsystems, the set value always applies starting from the uppermost Top model, i.e. if, as in the figure, the value 3 is set, then, regardless of whether a subsystem or a referenced model is located in the Top model, the respective system is generated up to a depth of 3.

To be able to calculate the coordinates of individual blocks/objects correctly, there must be a "real" block at the top left position in each system (no comment, no connecting line, etc.).



# Automation Academy

## Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom Learning



Virtual Classroom  
Learning



Online courses

## Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of training course content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

## Virtual classroom

**Remote Lectures** supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

## Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a targeted choice of information to be used as a reference.

## Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



**AutomationAcademy**









**B&R**  
**Industrial Automation GmbH**  
**A member of the ABB Group**  
B&R Straße 1  
5142 Eggelsberg, Austria  
[office@br-automation.com](mailto:office@br-automation.com)

t +43 7748 6586-0  
f +43 7748 6586-26

[br-automation.com](http://br-automation.com)

